SAPIENZA
UNIVERSITÀ DI ROMA

Motor Synergy Generalization Framework: New Targets in Multi-Finger In-Hand Manipulation

Faculty of Information Engineering, Informatics and Statistics
Artificial Intelligence and Robotics

Serena Trovalusci
ID number 2128733

Supervisor
Alessandro De Luca

Co-supervisor
Mitsuhiro Hayashibe

Academic Year 2025/2026

Thesis defended on May 21 2026
in front of a Board of Examiners composed by:

Prof. Alessandro De Luca (chairman)
Prof. Irene Amerini
Prof. Paolo Di Giamberardino
Prof. Giorgio Grisetti
Prof. Luca Iocchi
Prof. Domenico Lembo
Prof. Fabio Patrizi

Motor Synergy Generalization Framework: New Targets in Multi-Finger In-Hand Manipulation
Experimental Master's Thesis. Sapienza University of Rome

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: seretrovalusci@hotmail.com

To my family

Abstract

Dexterous in-hand manipulation with anthropomorphic robotic hands remains an open challenge in robotics. The high dimensionality of the action space, the complexity of finger-object contact dynamics, and the sparsity of informative feedback make reinforcement learning approaches sample-inefficient and slow to converge on 20-degree-of-freedom platforms such as the Shadow Dexterous Hand. Neuroscientific studies have shown that the human motor system manages this complexity through motor synergies, low-dimensional coordination patterns that are reused across a wide variety of tasks.

This research investigates whether motor synergies extracted from a single source manipulation task can accelerate reinforcement learning on a set of novel target tasks involving objects with substantially different geometries, without the need to re-fit the synergy basis.

We propose the Motor Synergy Generalization Framework, a three-stage pipeline. In Stage A, a full-DoF Soft Actor-Critic (SAC) agent is trained with Hindsight Experience Replay (HER) on a standard cube reorientation task until convergence. In Stage B, spatial synergies are extracted offline via Principal Component Analysis (PCA) from the joint-action trajectories of the converged policy. In Stage C, the resulting 5-dimensional synergy basis is frozen and used as a fixed linear decoder for training new agents on five object reorientation tasks: a small cube, a large cube, an egg, a cylinder, and a sphere.

The synergy-constrained agent consistently outperforms the full-action-space baseline across all five target tasks, achieving comparable or higher final success rates in substantially fewer environment timesteps and less wall-clock training time. The advantage is consistent regardless of object geometry, including objects whose shape was never encountered during synergy extraction.

Constraining the action space to a low-dimensional synergy subspace provides an effective way to improve the sample efficiency and reduce the training time of reinforcement learning in high-dimensional robotic manipulation.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

Robotic systems are playing an increasingly central role in modern society. From industrial assembly lines and warehouse automation to surgical assistance and rehabilitation devices, the ability to manipulate objects is fundamental to a wide range of applications. Current industrial and medical robots already achieve remarkable precision in structured, repetitive tasks. However, these systems rely on predefined trajectories, teleoperation, or carefully engineered control laws that assume accurate knowledge of the environment, and they lack the capacity to adapt autonomously to objects and situations that were not explicitly anticipated at design time.

The next frontier in robotics is therefore not precision in structured settings, which has largely been achieved, but *dexterous manipulation in unstructured environments*: the ability to reorient and manipulate arbitrary objects with the kind of fluid adaptability that humans exercise effortlessly every day. Achieving this with anthropomorphic robotic hands remains one of the most open and challenging problems in robotics [1].

1.2 The challenges of dexterous in-hand manipulation

In-hand manipulation refers to the ability to reorient and reposition an object within the hand without setting it down or using the other hand. It is one of the most fundamental yet complex fine motor capabilities in humans, requiring the coordinated control of multiple fingers, precise regulation of contact forces, and continuous integration of sensory feedback.

Replicating this task in robotic systems presents three main difficulties. The first concerns the high dimensionality of the control space. Anthropomorphic robotic hands, such as the Shadow Dexterous Hand, typically have 24 degrees of freedom (DoF), with 20 of them being independently actuated. This creates a large continuous action space where naive exploration is extremely inefficient and can lead to slow or unstable learning [10]. The second arises from the complexity of the contact dynamics between the fingers and the object: the interactions involve nonlinear friction, rolling and sliding contacts, and intermittent contact events that occur at high frequency and are difficult to capture in closed form. The third is partial

observability: even with tactile sensors, the full state of the hand-object system is uncertain, and the agent must infer contact forces and object pose from noisy, incomplete measurements [8, 16].

Classical model-based control approaches struggle to address these difficulties simultaneously. They rely on accurate analytical models of contact dynamics and explicit constraint formulations for grasp stability, assumptions that do not hold when dealing with unknown objects in real-time settings. Learning from demonstrations is an alternative, but high-quality human demonstrations of dexterous in-hand manipulation are expensive and difficult to obtain at scale, and policies trained on demonstrations tend to be brittle outside the demonstrated distribution [8].

1.3 Reinforcement learning for dexterous manipulation

Reinforcement learning (RL) offers a compelling alternative to model-based and demonstration-based approaches [10]. Rather than requiring an explicit model of the environment or a dataset of expert demonstrations, an RL agent learns manipulation policies directly from trial-and-error interaction, guided only by a reward signal that indicates whether the manipulation succeeded. This paradigm is a natural fit for dexterous manipulation, where the desired behaviour is hard to specify analytically but easy to evaluate.

Recent work has demonstrated that RL can produce impressive dexterous manipulation behaviour. Andrychowicz et al. [8] trained policies that perform vision-based object reorientation on a physical Shadow Dexterous Hand using domain randomisation, demonstrating that RL policies trained entirely in simulation can transfer to a real robot. Plappert et al. [16] established a suite of multi-goal in-hand reorientation benchmarks on which RL agents learn to reorient cubes, eggs, and pens to arbitrary target orientations.

However, applying RL to anthropomorphic hands remains challenging precisely because of the high dimensionality of the action space. With 20 or more degrees of freedom, random exploration covers the relevant region of the action space extremely slowly, and training requires enormous numbers of environment interactions. The sample inefficiency of standard RL in high-dimensional continuous action spaces is therefore the central bottleneck that limits its practical applicability to dexterous manipulation.

1.4 Motor synergies as a path to efficient learning

Dexterous hand manipulation is a redundant control task, as there are far more joints or muscles than necessary to achieve a given manipulation goal. This redundancy creates the so-called *paradox of redundancy* [3]: the same high dimensionality that makes exploration difficult and learning unstable for artificial agents can simultaneously serve as a fundamental mechanism for adaptability in biological systems. While an over-actuated control space complicates artificial learning by providing an explosion of possible solutions, it allows biological motor systems to remain robust and flexible in the face of system failures or unexpected environmental changes.

The human motor system manages the immense complexity of movement by organising groups of muscles and joints into coordinated activation patterns known as *motor synergies* [26, 35]. This strategy, first hypothesised by Bernstein, suggests that the central nervous system does not control every individual muscle or joint independently, but instead recruits a small number of functional modules. By searching in a significantly smaller space of synergy activations rather than in the full joint configuration space, the motor system drastically reduces the effective dimensionality of the control problem and enables the rapid acquisition of complex motor skills [7, 34].

An important property of these motor synergies is their reusability across tasks. Experimental studies on hand postures have shown that a small number of principal components can account for more than 80% of the variance in grasping a wide variety of everyday objects [34], suggesting that the motor system maintains a dictionary of reusable coordination primitives and learns new behaviours by recombining them [7, 26]. Recent computational work has validated this "transferable vocabulary" hypothesis: Kutsuzawa and Hayashibe [5] demonstrated that synergies extracted from planar reaching tasks can be recombined to reach new targets in different planes.

Finally, adopting a synergistic representation has been shown to enable remarkable levels of generalisation. Berg et al. [2] demonstrated that a Synergistic Action Representation (SAR) distilled from unsupervised "play" behaviours allows agents to manipulate novel objects with diverse geometries in a near zero-shot manner, whereas traditional end-to-end reinforcement learning fails to attain comparable transfer. By leveraging such structured manifolds, synergistic controllers offer a principled way to bridge the gap between structured robot precision and human-like fluid adaptability.

1.5 Contribution

This thesis investigates whether motor synergies extracted from a single source manipulation task can accelerate reinforcement learning on a set of novel target tasks involving objects with substantially different geometries.

We propose the *Motor Synergy Generalization Framework*, a three-stage pipeline in which a full-DoF Soft Actor-Critic (SAC) agent is first trained on a source task (Stage A), spatial synergies are then extracted offline via Principal Component Analysis (PCA) from the joint-action trajectories of the converged policy (Stage B), and the resulting synergy basis is finally used as a fixed low-dimensional action decoder for training new RL agents on target tasks (Stage C).

The specific contributions of this thesis are as follows.

- We propose and implement the Motor Synergy Generalization Framework, a three-stage pipeline that trains a full-DoF source policy (Stage A), extracts spatial synergies from its joint-action trajectories via PCA (Stage B), and reuses them as a fixed action decoder for novel target tasks on the Shadow Dexterous Hand (Stage C).

- We demonstrate that constraining the action space to a 5-dimensional synergy

subspace extracted from cube manipulation accelerates learning on a set of five target tasks spanning a wide range of object geometries: a smaller cube, a larger cube, an egg, a cylinder, and a sphere, in terms of both sample efficiency and wall-clock training time, with the magnitude of the advantage varying across object geometries.

- We show that the synergy advantage is consistent across all evaluated objects despite the synergy basis having been extracted from a single source task with a qualitatively different geometry from most of the targets.
- We provide a detailed analysis of the effect of object geometry on task difficulty and on the magnitude of the synergy advantage, identifying the conditions under which the transferred basis is most beneficial.

1.6 Thesis outline

The remainder of this thesis is organised as follows. Chapter 2 introduces the background material: reinforcement learning, motor synergies in neuroscience, the connection between biological and robotic synergies, Principal Component Analysis, and the simulation platform. Chapter 4 surveys the related literature and positions this work within it. Chapter 3 formalises the in-hand reorientation task as a multi-goal Markov Decision Process and defines the evaluation metrics. Chapter 5 describes the Motor Synergy Generalization Framework in detail. Chapter 6 presents the experimental results and their analysis. Chapter 7 draws conclusions and discusses directions for future work.

Chapter 2

Background

This chapter introduces the backbone of the thesis. The first pillar comes from machine learning: reinforcement learning, which is the algorithmic engine that allows an agent to learn how to behave by interacting with the environment when the only available feedback is a reward signal. The second pillar comes from neuroscience: the theory of motor synergies, which proposes that the complexity of biological movement is managed not by controlling every single muscle and joint independently but by combining a small set of coordinated activation patterns. This theory suggests that biological motor commands live on a low-dimensional manifold of coordinated activation patterns, and that learning new skills can be largely reduced to recombining a small dictionary of such patterns. The synergy hypothesis is the main source of inspiration for the method proposed in Chapter 5, which applies the same principle to the action space of a reinforcement-learning agent. This chapter also covers how the synergy concept has found its way into robotics, introduces the mathematical tool used to extract synergies from data, and introduces the simulation platform on which everything is implemented.

2.1 Key concepts in reinforcement learning

2.1.1 From biological to artificial learning

A natural starting point to understand reinforcement learning is to observe how learning occurs in nature. A child who learns to walk does not read a textbook on bipedal locomotion, nor is it provided with explicit kinematic targets for every joint of its body. Instead, the child interacts with the environment: it pushes with its legs, falls, observes the outcome, and gradually refines its movements. Over weeks of trial and error, a robust policy emerges that adapts to uneven floors, different shoes, and the presence of obstacles. The same principle governs countless other everyday skills: a toddler babbling its way into fluent speech or a teenager learning to ride a bicycle. In every case, the learner improves not by being told what to do, but by experiencing the consequences of their own actions. As Sutton and Barto observe in the opening pages of their textbook, “learning from interaction is a foundational idea underlying nearly all theories of learning and intelligence” [17].

Reinforcement Learning (RL) is the computational framework that formalises this paradigm [17]. An RL agent learns by interacting with an environment: at every

time step it observes a state, selects an action, and receives a scalar reward signal that indicates the desirability of the resulting state. The agent's objective is to discover an optimal policy, which determines a mapping from states to actions that maximises the cumulative reward obtained over time. Unlike supervised learning, RL does not assume access to a labelled dataset of correct decisions; unlike unsupervised learning, it does not attempt to learn the latent structure of a fixed dataset. The only feedback available to the agent is the reward signal, which makes RL a natural candidate for problems where the desired behaviour is hard to specify but easy to evaluate. This is the case for dexterous manipulation, where contact dynamics are discontinuous and notoriously difficult to model in closed form. Classical model-based approaches would require an accurate analytical model of the finger-object contact dynamics, explicit constraint formulations for the grasp stability conditions, and carefully engineered cost functions to handle the continuous sequence of contact events. RL sidesteps all of this: the only thing that needs to be specified is a reward signal indicating whether the manipulation succeeded or not. This is a powerful idea that has been successfully applied to a wide variety of problems, from playing Atari games [20] to in-hand dexterous manipulation [8], and is the algorithmic backbone of this thesis.

2.1.2 Markov decision processes

The standard mathematical model of the RL problem is the Markov Decision Process (MDP), a compact formalisation of sequential decision-making under uncertainty [17]. An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ where

- $\mathcal{S}$ is the set of possible states of the environment;
- $\mathcal{A}$ is the set of actions available to the agent;
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition function, with $P(s' \mid s, a)$ the probability of transitioning to a new state s' given that the agent took action a in state s;
- $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function associated with a state-action pair;
- $\gamma \in [0, 1)$ is a discount factor that weights the importance of future rewards.

The defining property of an MDP is the *Markov property*, which states that the distribution of the next state depends only on the current state and the taken action, not on the history of past interactions. This property is a remarkably useful modelling choice as it means that the agent can decide its next action looking only at the current state, without having to remember the entire past. Formally, the search for an optimal policy can be restricted to policies of the form $\pi(a \mid s)$, rather than policies that depend on the whole history of states and actions.

The agent's behaviour is described by a policy probability function $\pi(a \mid s)$ that assigns a probability to each action in each state. The interaction between agent and environment generates a trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$ whose law is jointly determined by the policy and by the transition dynamics. The agent's objective is to find a policy $\pi^\star$ that maximises the expected discounted return

$$J(\pi) \;=\; \mathbb{E}_{\tau \sim \pi}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]. \tag{2.1}$$

The discount factor γ balances immediate and future rewards: smaller values prioritise short-term gains, while values close to one make the agent take long-term outcomes into account.

2.1.3 Value functions and the Bellman equations

In reinforcement learning algorithms, the concept of value functions and the Bellman equations provides the mathematical foundation for estimating long-term rewards and determining optimal behaviour [17].

The *state-value function* of state s under a policy π is defined as the expected cumulative discounted reward obtained by starting in s and following π thereafter:

$$V^{\pi}(s) \;=\; \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty}\gamma^{k} r_{t+k} \;\middle|\; s_t = s\right]. \tag{2.2}$$

Intuitively, $V^{\pi}(s)$ does not measure the reward of s itself but the worth of s as a *starting point* for the policy π.

The *action-value function*, or *Q-function*, represents the expected cumulative discounted reward starting from s, performing action a, and thereafter following policy π:

$$Q^{\pi}(s,a) \;=\; \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty}\gamma^{k} r_{t+k} \;\middle|\; s_t = s,\; a_t = a\right]. \tag{2.3}$$

Both functions satisfy recursive identities, known as the *Bellman expectation equations*, that follow from the Markov property and express the value of a state (or state-action pair) as the expected immediate reward plus the discounted value of the successor:

$$V^{\pi}(s) \;=\; \mathbb{E}_{a\sim\pi,\, s'\sim P}\big[\, R(s,a) + \gamma\, V^{\pi}(s')\big]\,. \tag{2.4}$$

Equation (2.4) evaluates a *given* policy π. To find the *best* policy, we need a different equation. The key observation is that (2.4) can be rewritten as

$$V^{\pi}(s) \;=\; \mathbb{E}_{a\sim\pi}[Q^{\pi}(s,a)]\,, \tag{2.5}$$

that is, the value of a state is the expected action-value when actions are sampled from π. Now consider what an optimal policy $\pi^{\star}$ must look like: in every state, it should always choose the action with the highest action-value, never wasting probability on suboptimal actions. This means that for an optimal policy, the expectation over $a \sim \pi$ can be replaced by a maximum over a, which turns the recursion into the *Bellman optimality equation*:

$$V^{\star}(s) \;=\; \max_{a\in\mathcal{A}}\; \mathbb{E}_{s'\sim P}\big[\, R(s,a) + \gamma\, V^{\star}(s')\big]\,, \tag{2.6}$$

or, equivalently, in action-value form,

$$Q^{\star}(s,a) \;=\; \mathbb{E}_{s'\sim P}\Big[\, R(s,a) + \gamma \max_{a'\in\mathcal{A}} Q^{\star}(s',a')\Big]\,. \tag{2.7}$$

A direct consequence is that $Q^{\star}$ alone is enough to act optimally: choosing $\arg\max_a Q^{\star}(s,a)$ in every state recovers an optimal policy, with no need for a model of the environment.

2.1.4 Classes of reinforcement-learning algorithms

RL algorithms can be broadly classified along three axes.

Model-based vs. model-free. Model-based methods attempt to learn a model of the transition dynamics P and of the reward function R, and then plan using this model. Model-free methods, in contrast, learn value functions and policies directly from sampled interactions, without ever building an explicit model of the environment. Given the hybrid and discontinuous contact dynamics of dexterous manipulation, a model-free approach is the natural choice.

On-policy vs. off-policy. On-policy methods evaluate and improve the same policy that is used to collect data: every time the policy is updated, the previously collected trajectories become obsolete and new ones must be gathered. Off-policy methods, instead, can reuse past experience stored in a replay buffer regardless of which policy collected it. This allows them to be far more sample-efficient, a critical advantage in robotic tasks where every interaction is computationally expensive.

Value-based, policy-based, and actor–critic. Value-based methods focus all their effort on estimating an action-value function $Q(s, a)$; the policy is never represented explicitly, but is recovered on the fly by acting greedily with respect to the learned Q. Q-learning, together with its deep counterpart DQN [20], is its canonical incarnation. The weakness is that the implicit greedy step requires a $\max_a Q(s, a)$ at every decision, which becomes problematic in continuous action spaces. Policy-based methods take the opposite stance: they parameterise the policy $\pi_\theta(a \mid s)$ directly as a differentiable function and optimise its parameters by gradient ascent on the expected return $J(\pi_\theta)$. They are naturally suited to continuous actions and to stochastic policies, but they tend to be sample-inefficient because they cannot easily reuse old data. Actor–critic methods combine the two perspectives in a single algorithm: a *critic* learns a value function in the spirit of value-based methods, while an *actor* learns an explicit policy in the spirit of policy-based methods, and the two networks help each other. The critic provides low-variance gradient estimates to the actor, and the actor produces the trajectories on which the critic is trained. This family of algorithms, which includes DDPG [19], TD3 [12] and SAC [13], is the standard choice for continuous high-dimensional control problems such as dexterous manipulation.

The algorithm adopted in this thesis, Soft Actor–Critic (SAC) [13], sits at the intersection of all three classifications. It is model-free, meaning it learns directly from interactions without ever building an explicit model of the contact dynamics. It is off-policy, meaning it can reuse past experience stored in a replay buffer, which is critical for sample efficiency in a computationally expensive simulation environment. And it is an actor–critic method, meaning it maintains both an explicit stochastic policy and a learned Q-function that provides stable gradient estimates for the policy update. As we will see in the next section, SAC further extends this framework by adding an entropy maximisation objective that encourages exploration, making it particularly well suited to high-dimensional continuous control problems such as dexterous in-hand manipulation.

2.1.5 Soft Actor–Critic and maximum entropy RL

The classical RL objective (2.1) considers only the expected return, without encoding any preference over the stochasticity of the policy. SAC [13] extends this by adding an entropy bonus to the objective that rewards the policy for remaining as stochastic as possible while still achieving high return, as formalised in (2.8) below. This is particularly relevant in high-dimensional continuous action spaces like the one studied in this thesis, where without the entropy bonus the policy would tend to collapse prematurely to a suboptimal strategy, failing to explore the full range of coordination patterns necessary for successful manipulation.

$$J_{\mathrm{MaxEnt}}(\pi) \;=\; \mathbb{E}_{\pi}\left[\sum_{t} \gamma^{t}\left(R(s_t, a_t) + \alpha\, \mathcal{H}\big(\pi(\cdot \mid s_t)\big)\right)\right], \tag{2.8}$$

where $\mathcal{H}$ denotes Shannon entropy and $\alpha > 0$ is a temperature that trades off reward against randomness. Maximum-entropy RL has three appealing properties: (i) it encourages exploration by preventing premature collapse to a single action mode; (ii) it produces policies that are robust to perturbations in the reward or dynamics; and (iii) it admits a family of consistent Bellman-like updates for the corresponding *soft* value functions.

SAC maintains a stochastic actor π_θ that outputs the mean and variance of a Gaussian distribution over actions, and two independent critic networks Q_{ϕ_1}, Q_{ϕ_2}, whose minimum is used in the target update to mitigate the over-estimation bias that afflicts single-critic methods. In its auto-tuned variant [14], the temperature α is itself a learnable parameter, adjusted so that the policy entropy tracks a target value: this removes one of the most sensitive hyperparameters to tune and stabilises training across tasks with very different reward scales.

2.1.6 Multi-goal RL and Hindsight Experience Replay

The in-hand manipulation problem studied in this thesis is formulated as a *multi-goal* problem: rather than learning to reach a single fixed target configuration, the agent must learn a policy $\pi(a \mid s, g)$ that generalises across a continuous space of goal poses $g \in \mathcal{G}$, each sampled independently at the beginning of every episode. The natural extension of the MDP formalism to this setting is the *multi-goal MDP* [21], described by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{G}, P, R, \gamma)$, in which the reward $R(s, a, g)$ and the policy $\pi(a \mid s, g)$ are explicitly conditioned on a goal $g \in \mathcal{G}$. Universal Value Function Approximators (UVFA) [21] formalise this setting by learning a single neural network that approximates $Q(s, a, g)$ across the whole goal space, thereby enabling generalisation from goals observed during training to unseen goals at test time.

The most natural reward structure for this setting is a sparse binary signal: the agent receives a reward of 0 if the goal has been reached within a given tolerance and -1 otherwise. This choice is deliberately preferred over shaped alternatives, which require significant domain expertise to design and can inadvertently bias the policy towards suboptimal strategies. The price of this formulation is that the reward signal becomes extremely sparse: in high-dimensional problems, random exploration policies almost never reach the goal, so almost every trajectory carries

no useful learning signal and gradient-based learning stalls. Andrychowicz *et al.* [18] illustrate the difficulty with a deliberately stripped-down example, the bit-flipping environment: the agent must turn a binary string of length n into a target string by flipping one bit at a time, receiving a reward of -1 at every step until the goal is reached. With a vanilla DQN agent, learning succeeds for $n \leq 13$ and fails outright once $n > 13$; the same agent equipped with their proposed relabelling scheme solves the problem for $n > 50$. The example crystallises a general phenomenon: as soon as the goal manifold becomes a vanishingly small subset of the state space, no amount of exploration noise can compensate for the absence of an informative learning signal.

Hindsight Experience Replay (HER) is the technique introduced by Andrychowicz *et al.* [18] to address this difficulty. The core idea is the *relabelling* of failed trajectories: after an episode in which the agent fails to reach the intended goal g, HER stores in the replay buffer additional copies of each transition in which g has been replaced by one or more goals that were actually achieved during the same episode (Figure 2.1). Because, by construction, the relabelled goals were reached, each relabelled transition yields a non-zero reward. The agent thus learns from its mistakes as if they were successes for a different task, and progressively bootstraps its way towards solving the original goals. The resulting effect is an *implicit curriculum*: the achieved goals at the start of training are usually close to the initial state, so the policy first learns to reach nearby targets, and as it improves the relabelled goals become increasingly distant and varied. HER is moreover *algorithm-agnostic*: it operates purely at the level of the replay buffer and is therefore compatible with any off-policy RL method.

Four strategies exist for choosing the hindsight goals associated with a transition (s_t, a_t, s_{t+1}, g) collected during an episode of length T. The *final* strategy reuses only the achieved goal of the last state s_T. The *future* strategy samples k additional goals uniformly from the future states $\{s_{t+1}, \ldots, s_T\}$ of the same episode. The *episode* strategy samples k goals from all states of the episode, and the *random* strategy samples k goals from any episode in the replay buffer. The *future* strategy is the most consistently effective, because it yields a natural curriculum of progressively more ambitious sub-goals. The present thesis adopts the *future* strategy: the combination of SAC with HER is the backbone of the reinforcement-learning component of this work, and is described algorithmically in Chapter 5.

2.2 Motor synergies in neuroscience

This section reviews the experimental and theoretical foundations of the motor synergy hypothesis, from its original formulation by Bernstein to its modern computational interpretations.

2.2.1 Bernstein's degrees-of-freedom problem

The foundational formulation of the motor-control problem still discussed today is attributed to the Russian physiologist Nikolai Bernstein [35]. Bernstein observed that the human body comprises hundreds of joints and thousands of muscles. At any instant, the central nervous system faces the task of choosing, among the enormous number of joint-muscle configurations, the ones compatible with a given task. How does the nervous system solve such a highly redundant, high-dimensional

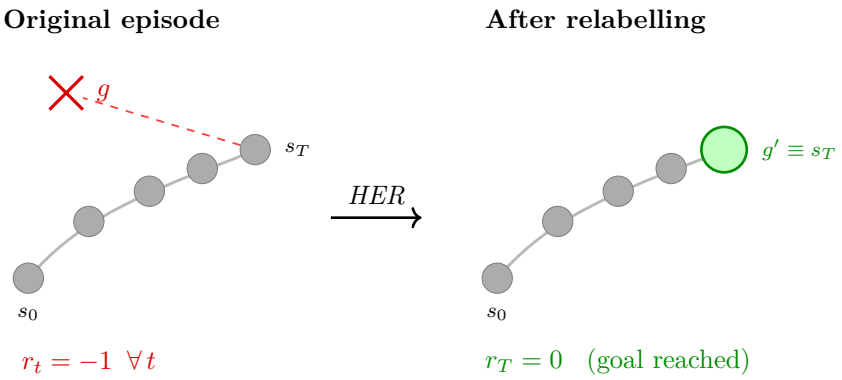


Figure 2.1 Hindsight Experience Replay (HER) relabelling. In the original episode (left), the agent does not reach the target goal g and receives $r_t = -1$ at every timestep, providing no positive learning signal. HER replaces the original goal with the actually achieved final state $g' \equiv s_T$, turning the same trajectory into a successful experience (right) with $r_T = 0$.

control problem in real time, without any apparent effort, in a way that is robust to perturbations and transferable across tasks?

Bernstein's theory is that the motor system does not control every degree of freedom independently. Rather, it restructures the problem: large groups of muscles and joints are coupled into functional units which are controlled as a whole, thereby reducing the dimensionality of the effective control space. What later became known as the *motor synergy hypothesis* is the modern articulation of this intuition. A motor synergy is any fixed (or slowly varying) coupling pattern between the degrees of freedom of the body. A complete movement is then produced by linearly combining a small number of such patterns, each weighted by a time-dependent activation.

Formulated this way, the hypothesis has two important implications. First, if the hypothesis is correct, the space of motor commands actually produced by a behaving organism should be low-dimensional: most of its variance should be captured by a small number of components. Second, it provides a concrete computational strategy for controlling a redundant body: instead of searching in the original high-dimensional space of all possible motor commands, the controller only needs to search in the much smaller space of synergy activations, letting the synergies themselves map the result back into full motor commands.

2.2.2 Muscle and kinematic synergies

The literature distinguishes two complementary levels at which synergies can be defined and measured: the *muscle* level and the *kinematic* level.

Muscle synergies describe coordinated activation patterns of groups of muscles. The experimental signal here is surface electromyography (EMG): a set of electrodes placed on the skin records, simultaneously and at every instant, the level of electrical activity of each muscle of the limb under study. The result of a single trial is a high-dimensional time series, one curve per muscle.

A long line of experimental work has shown that muscle activation patterns recorded during natural behaviours can be reconstructed as a sum of a small number of coordinated components. In a seminal study on freely moving frogs, d'Avella *et al.* [32] introduced *time-varying* muscle synergies, spatiotemporal activation patterns with fixed temporal profiles that can be independently scaled in amplitude and shifted in time. Recording EMG from thirteen hindlimb muscles across behaviours such as kicking, jumping, swimming and walking, they showed that combinations of just three such synergies account for the variety of muscle patterns required to kick in different directions, and that at least one kicking synergy is shared across behaviours, suggesting a small dictionary of motor primitives.

This experimental literature raised an important methodological question: are muscle synergies genuine physiological structures or artefacts of a particular decomposition algorithm? Tresch, Cheung and d'Avella [30] addressed this by systematically comparing several matrix factorisation methods, including PCA, FA, ICA, NMF and pICA on simulated datasets with known ground truth and on experimental EMG recordings. They found that the most robust and accurate algorithms were ICAPCA (ICA applied to the PCA subspace) and probabilistic ICA (pICA). PCA alone was effective at identifying the correct low-dimensional subspace, but not at recovering the individual synergies.

More recent reviews consolidate the case that muscle synergies are neurophysiological entities rather than purely biomechanical conveniences. Bizzi and Cheung [23] survey evidence for spinal modules in frogs and rats, cortical microstimulation in monkeys that evokes naturalistic reach-and-grasp synergies, and clinical data from stroke survivors in whom impaired movements still appear to be composed of largely intact synergies. Together these findings support the view that a small repertoire of muscle synergies, likely originating from circuits in the spinal cord, regulates a wide variety of natural behaviours while reducing the effective degrees of freedom faced by the controller.

It is important to note that these conclusions concern the identification of *muscle* synergies from noisy EMG signals, where the goal is to recover underlying physiological modules. In that setting, PCA is a useful tool for locating the low-dimensional subspace but is not the best choice for isolating individual synergies.

Kinematic synergies describe coordinated patterns of *joint-angle* variation, rather than of muscle activation. The paradigmatic study here is the work of Santello, Flanders and Soechting on hand postures [34]. The authors recorded the configuration of the 15 joints of the human hand while subjects shaped their hand to grasp a wide variety of everyday objects (pens, coins, cups, tools). Applying Principal Component Analysis to the resulting postural data, they showed that the first two principal components alone already capture more than 80% of the variance, and that five components are sufficient to reconstruct the data almost exactly. Remarkably, the first component reflects a coordinated flexion of all fingers while the second captures the opposition of the thumb, two patterns that are directly interpretable in terms of the functional role of the hand. The Santello study is, to this day, the reference point for almost every synergy-based analysis of the hand.

Muscle and kinematic synergies are not in competition; they quantify the same

underlying phenomenon at different levels of the motor hierarchy: dimensionality reduction in motor control. For the purposes of this thesis, the relevant level is the kinematic one. The Shadow Dexterous Hand is actuated at the *joint* level by position commands, not at the muscle level by tendon tensions. Its natural analogue of a Santello synergy is a coordinated pattern of joint commands, which is precisely the object extracted by the method described later in this thesis.

2.2.3 Mathematical forms of motor synergies

Beyond the muscle–kinematic distinction, the synergy literature distinguishes three complementary mathematical ways of parameterising a low-dimensional motor vocabulary:

Time-invariant (spatial) synergies. The simplest and most widely used formulation assumes that every motor command $x(t) \in \mathbb{R}^m$ is a linear combination of K fixed activation patterns $w_k \in \mathbb{R}^m$, scaled by time-varying scalar coefficients $c_k(t)$:

$$x(t) \;\approx\; \sum_{k=1}^{K} c_k(t)\, w_k, \qquad K \ll m. \tag{2.9}$$

Here the *spatial* structure of the synergies w_k is fixed once and for all, while their temporal profiles $c_k(t)$ are left free for the controller to specify at every instant. This is the class of synergies identified by Santello *et al.* [34] for the hand, and they are typically extracted using Principal Component Analysis (PCA), which is described in Section 2.5.

Time-varying (temporal) synergies. A formulation introduced by d'Avella and collaborators for reaching EMG data [32]. It assigns each synergy a fixed temporal profile $\phi_k(t) \in \mathbb{R}^m$ and models an entire movement as a time-shifted linear superposition of such profiles:

$$x(t) \;\approx\; \sum_{k=1}^{K} h_k\, \phi_k(t - t_k), \tag{2.10}$$

where h_k is a scalar amplitude and t_k a scalar onset time. The primitives ϕ_k here play the role of rhythmic or discrete movement templates and the controller acts by selecting (h_k, t_k) for each primitive rather than a full trajectory. The price of the extra expressiveness is that ϕ_k cannot be extracted by PCA: specialised algorithms such as Non-Negative Matrix Factorisation or shift-NMF are required.

Spatio-temporal synergies. Kutsuzawa and Hayashibe [5] adopt an intermediate representation in which each synergy $w_\ell(t) \in \mathbb{R}^m$ is a full joint-torque trajectory of fixed shape, and a movement is reconstructed as a scalar-weighted sum

$$x(t) \;\approx\; \sum_{\ell=1}^{L} h_\ell\, w_\ell(t). \tag{2.11}$$

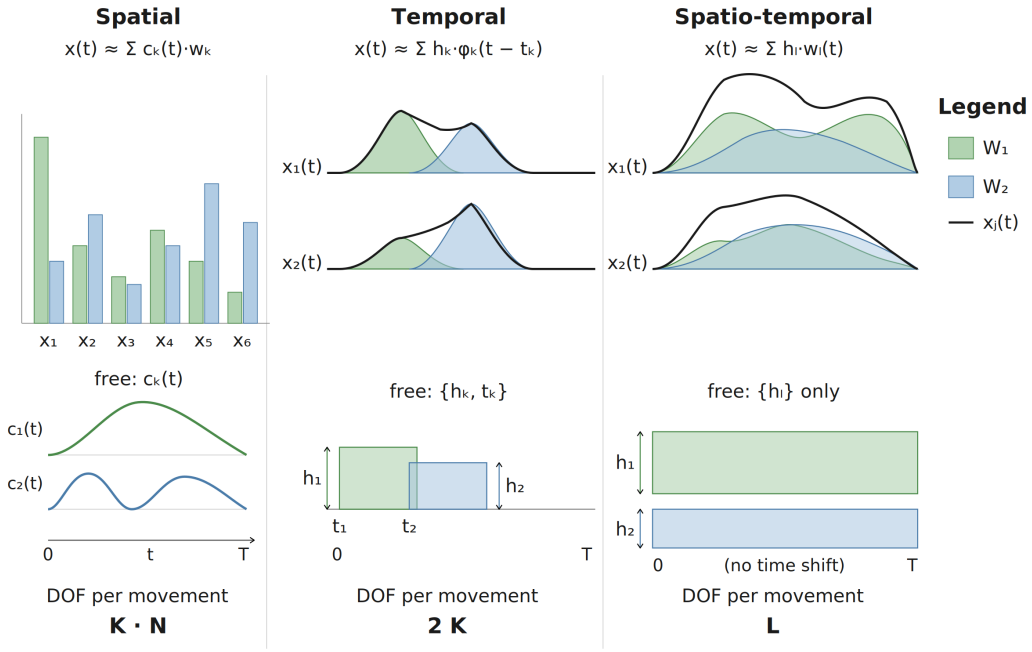


Figure 2.2 The mathematical forms of motor synergies. In each panel, gray indicates fixed components and purple indicates free parameters. *Spatial*: the spatial pattern w_k is fixed; the temporal activation $c_k(t)$ is free at every time step, so each joint can follow any temporal profile. *Temporal*: the template shape φ_k is fixed; the onset time t_k and amplitude h_k are free per movement. *Spatio-temporal*: the entire trajectory $w_\ell(t)$ is fixed; only the scalar amplitude h_ℓ is free. This thesis adopts the spatial formulation (2.9), which imposes the weakest assumption and leaves the reinforcement-learning agent free to choose the temporal activations at every control step.

Compared with (2.9), the activations h_ℓ are reduced from time-varying signals to a small vector of scalars, which makes the control problem tractable by black-box optimisation (e.g. CMA-ES) but restricts the controller to producing a small catalogue of pre-shaped trajectories.

The three formulations (2.9)–(2.11) encode progressively stronger assumptions: spatial synergies constrain only the instantaneous command structure; temporal synergies add a fixed time profile; spatio-temporal synergies freeze the full trajectory shape. Figure 2.2 summarises the three representations and the degree of freedom available to the controller in each case. The present thesis commits to the weakest of the three assumptions (2.9): joint-level synergies are spatial patterns, and the reinforcement-learning agent retains full freedom in choosing their temporal activations.

2.2.4 Shared and task-specific synergies

A property of synergies that becomes central for the present work is their *reusability across tasks*. Experimental evidence suggests that the same synergy structures recur across related tasks with only minor adaptation. Santello, Flanders and Soechting [34] showed that the same small set of principal components accounts for hand postures

across a wide variety of grasping tasks, and d'Avella and Bizzi [31] demonstrated that muscle synergies are shared across different natural motor behaviors while only a subset are task-specific. These findings suggest that the motor system maintains a dictionary of reusable coordination primitives and learns new behaviours largely by recombining them, rather than learning an entirely new set of commands for each task.

Computational studies have begun to test this idea in simulated musculoskeletal models. Rückert and d'Avella [24] implemented a shared-synergy structure based on dynamic movement primitives (DMP) and showed that the resulting representation accelerated learning. Al Borno, Hicks and Delp [7], working with a physiologically realistic upper-extremity model with 47 muscles, reported that a small repertoire of motor modules suffices to drive a wide range of reaching tasks and improves both learning speed and generalisation to new targets, providing one of the cleanest computational demonstrations that extracted synergies are not merely a redescription of the training data but genuinely transfer to new tasks. Kutsuzawa and Hayashibe [5] extended this line of evidence to reinforcement learning using a full 7-DoF arm model: they showed that spatio-temporal synergies extracted from policies trained on horizontal and sagittal reaching can be combined to generalize to entirely new planes that neither repertoire could reach independently. Finally, at the muscle level, Berg *et al.* [2] reported the most striking generalisation result so far: a Synergistic Action Representation (SAR) distilled from a play-phase policy on simple tasks generalises zero-shot to in-hand reorientation of a hundred novel objects and to locomotion on varied terrains, while end-to-end reinforcement-learning baselines with the same sample budget fail on both.

All of these results converge towards the same message: a properly chosen synergy basis is not a task-specific compression technique but a transferable motor vocabulary. This thesis tests whether the same message holds at the level of kinematic synergies extracted from a *single* joint-controlled task on an anthropomorphic hand, by evaluating whether a fixed low-dimensional synergy basis accelerates policy learning on a set of novel object manipulation tasks that were never seen during synergy extraction.

2.3 Redundancy resolution in classical robotics

It is useful to recall how the problem of controlling an overactuated manipulator has been treated within classical robotics before explaining why a data-driven alternative is needed for the present thesis. The dominant framework is the theory of *kinematic redundancy* [29]. Consider a manipulator with joint-configuration vector $q \in \mathbb{R}^n$ and task-space vector $\xi \in \mathbb{R}^r$, related by a non-linear kinematic map $\xi = k(q)$. Differentiating gives the differential kinematic relation $\dot{\xi} = J(q)\,\dot{q}$, where $J(q) \in \mathbb{R}^{r\times n}$ is the geometric Jacobian. When $n > r$, the manipulator is said to be *kinematically redundant*: the task space does not uniquely determine the joint velocities, so the controller must choose one inverse among the infinitely many that realise the desired end-effector motion. A standard choice is the pseudoinverse solution

$$\dot{q} \;=\; J^{\dagger}(q)\,\dot{\xi} \;+\; \left(I_n - J^{\dagger}(q)\,J(q)\right)\dot{q}_0, \tag{2.12}$$

where $J^\dagger$ is the Moore–Penrose pseudoinverse and the operator $(I_n - J^\dagger J)$ projects an arbitrary velocity $\dot{q}_0$ onto the *null space* of J. The first term realises the task; the second exploits the redundancy to satisfy secondary objectives such as joint-limit avoidance or manipulability maximisation, without interfering with the primary task. This basic construction has been refined along several axes in the classical robotics literature, addressing joint limits, obstacle avoidance, and multiple ranked tasks.

However, two fundamental difficulties make the classical approach ill-suited to the in-hand manipulation setting studied in this thesis. First, equation (2.12) reasons about instantaneous velocities through $J(q)$, requiring a reliable analytical kinematic model of the task at every control step. The synergies extracted in this thesis are instead *postural* entities defined in joint-position space; they do not arise naturally from a Jacobian decomposition of a specific task. Second, and more fundamentally, in-hand manipulation is governed by a complex, high-frequency sequence of rolling, sliding, and breaking contacts between the fingers and the object. The resulting contact dynamics are non-smooth and highly nonlinear, so the task-space map $\xi = k(q)$ is neither smooth nor differentiable in any useful sense during manipulation. Without a well-defined $J(q)$, the entire machinery of pseudoinverse projection and null-space correction breaks down.

Motor synergies offer a data-driven alternative that sidesteps both difficulties. Rather than resolving redundancy through an analytical kinematic model, one constrains all joint commands to lie in a low-dimensional subspace estimated from trajectories that the robot has actually executed. The subspace is defined by the covariance structure of the commands themselves, and therefore remains well-defined even in the contact-rich regime where classical redundancy resolution fails. The mathematical tool used to extract this subspace is Principal Component Analysis, described in Section 2.5.

2.4 From biological to robotic synergies

The neuroscientific literature of Section 2.2 has gradually found its way into robotics, where it has evolved from a source of biological inspiration into a completely developed design principle for the control of highly redundant hands. The most influential line of work is the one initiated by Ciocarlie and Allen [28], who transported Santello's postural synergies into autonomous grasp planning. Their idea was to reduce the dimensionality of the grasp-planning search space by projecting candidate hand configurations onto the first two postural synergies identified by Santello et al.: a task that would otherwise require searching a 15- or 20-dimensional configuration space becomes a search in a two-dimensional synergy plane, and is solved more efficiently with essentially no loss in grasp quality. This result is the cleanest demonstration of the computational payoff of synergies in robotics.

In parallel, Bicchi and collaborators developed the notion of *soft synergies* and its mechanical counterpart, *synergy-based underactuation* [26]. Rather than enforcing synergies in software, as Ciocarlie and Allen do, the soft-synergy approach embeds them into the physical design of the hand through compliant tendon-pulley couplings that mechanically realise the first few principal synergies of a target repertoire. The result is a family of low-cost, highly adaptive hands: the Pisa/IIT SoftHand [22]

being the most widely known, a hand that can grasp an impressive variety of objects despite having only a single motor. Soft synergies confirm the practical power of the synergy abstraction by demonstrating that it can be compiled directly into hardware.

The lines of work described so far share a common assumption: the synergy basis is either measured on human subjects or designed by hand, and is then used as a fixed prior for control or mechanical design. The question of whether a synergy basis can be *learned* automatically, and whether it can be integrated inside a data-driven controller, emerged naturally with the rise of reinforcement learning in robotics. Hayashibe and Shimoda introduced the *synergetic learning control* paradigm [15], in which a low-dimensional synergy basis is treated as a redundancy-resolution device that coordinates the joints of an over-actuated robot through a small number of virtual command channels. Building on this, Chai and Hayashibe [9] asked whether high-performing deep reinforcement-learning policies *spontaneously* develop motor synergies, showing that the answer is broadly yes and that the degree of synergy structure correlates with task performance. This is the first explicit bridge between the synergy hypothesis of motor neuroscience and modern policy-gradient methods.

The present work sits at the intersection of these traditions. Like Santello et al. and like Ciocarlie and Allen, this thesis studies postural-level kinematic synergies extracted via Principal Component Analysis. Following the line of work initiated by Hayashibe and collaborators, we learn these synergies from the trajectories of a reinforcement-learning agent rather than from human data, and we demonstrate that a fixed synergy basis extracted from one task accelerates learning on a family of novel manipulation tasks with different object geometries.

2.5 Principal Component Analysis

Principal Component Analysis (PCA) is a dimensionality-reduction technique that maps a high-dimensional dataset onto a lower-dimensional one while preserving as much of the variance in the data as possible. The *principal components* are chosen so as to capture the largest residual variance under the constraint of orthogonality to the previous ones. Equivalently, the first K principal components span the K-dimensional linear subspace that minimises the mean-squared reconstruction error of the original variables. In this thesis, PCA is used as the extraction algorithm for the spatial synergies of equation (2.9): the principal components recovered from a set of joint-command vectors are identified with the synergy patterns w_k, and the corresponding scores with the activations c_k.

2.5.1 Derivation and variance-maximisation

Let $X \in \mathbb{R}^{n \times M}$ be a data matrix whose n rows are samples and whose M columns are features. Let $\bar{x} \in \mathbb{R}^M$ denote the column-wise mean of X, and let $\tilde{X} = X - \mathbf{1}_n \bar{x}^\top$ be the centred data. The empirical covariance matrix is

$$\Sigma \;=\; \frac{1}{n-1}\,\tilde{X}^\top \tilde{X} \;\in\; \mathbb{R}^{M \times M}. \tag{2.13}$$

PCA seeks a set of $K \leq M$ orthonormal directions that, taken together, capture as much of the variance of the data as possible. Formally, one asks for a matrix

$W \in \mathbb{R}^{K\times M}$ with orthonormal rows ($WW^\top = I_K$) that maximises the *explained variance*

$$W^\star \;=\; \arg\max_{WW^\top = I_K} \operatorname{tr}\!\left(W\,\Sigma\,W^\top\right). \tag{2.14}$$

The solution is a classical consequence of the spectral theorem: the rows of $W^\star$ are the eigenvectors of Σ associated with its K largest eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_K$, and the corresponding explained variance equals $\sum_{k=1}^{K} \lambda_k$. Equivalently, $W^\star$ is obtained from the singular value decomposition $\tilde{X} = U\Lambda V^\top$ by retaining the first K rows of $V^\top$, a form that is numerically more stable and is the one actually used in the implementation.

Given $W^\star$, the data are factorised as

$$X \;\approx\; \mathbf{1}_n \bar{x}^\top + H\,W^\star, \qquad H \;=\; \tilde{X}\,(W^\star)^\top \;\in\; \mathbb{R}^{n\times K}. \tag{2.15}$$

The two matrices $W^\star$ and H play conceptually distinct roles. Each row of $W^\star \in \mathbb{R}^{K\times M}$ is one *synergy pattern*: a vector of length M that encodes a coordinated template of joint displacements, answering the question of *what* the coordination pattern looks like. Each row of $H \in \mathbb{R}^{n\times K}$ is the *activation vector* for one observation: the k-th entry h_{ik} is a scalar that quantifies how much of synergy k is present in sample i, answering the question of *how much* of each template is recruited at that instant. The reconstruction of a single sample therefore reads

$$x_i \;\approx\; \bar{x} \;+\; \sum_{k=1}^{K} h_{ik}\, w_k^\star, \tag{2.16}$$

where $w_k^\star$ is the k-th row of $W^\star$. This is the discrete-time counterpart of the spatial synergy model of equation (2.9): the fixed synergy patterns $w_k^\star$ play the role of the basis vectors w_k, and the scalar activations h_{ik} play the role of the time-varying coefficients $c_k(t)$. The controller retains full freedom to choose the activations; the synergy patterns themselves are fixed after extraction.

2.5.2 Choice of the number of components

The most important hyperparameter of PCA is the number of retained components K. The natural criterion is the *reconstruction quality* on a held-out portion of the data, measured by the coefficient of determination

$$R^2(K) \;=\; 1 \;-\; \frac{\sum_i \|x_i - \hat{x}_i^{(K)}\|^2}{\sum_i \|x_i - \bar{x}\|^2}, \tag{2.17}$$

where $\hat{x}_i^{(K)}$ is the reconstruction of x_i from the first K synergies. However, R^2 evaluates the synergies purely *reconstructively*, asking whether the original commands can be approximated by the basis. It says nothing about whether the target task can actually be solved by acting in the synergy subspace. The correct way to choose K is therefore task-driven rather than reconstruction-driven: the number of components is determined by combining the R^2 curve with direct task-performance evaluation, retaining the smallest K for which the synergy-constrained agent still achieves meaningful learning progress. This follows the same joint reconstruction-and-performance criterion adopted by Kutsuzawa and Hayashibe [5]. The specific value chosen and its justification are reported in Section 5.3.3 of Chapter 5.

2.5.3 Why PCA over alternative factorisations

Two main alternatives to PCA appear in the synergy literature: Non-Negative Matrix Factorisation (NMF) and Independent Component Analysis (ICA).

NMF, used by Kutsuzawa and Hayashibe [5] to extract spatio-temporal synergies from joint-torque trajectories, requires non-negative inputs. Applying it to joint positions would therefore demand splitting each joint into positive and negative components and stacking them, doubling the feature dimension from M to $2M$. In preliminary experiments with this parametrisation, training became noticeably unstable and the decoded actions frequently left the environment's nominal action range, so hard clipping had to be applied at every step. This combination of increased dimensionality and aggressive clipping is a plausible explanation for the poor performance of NMF-based synergies in our setting.

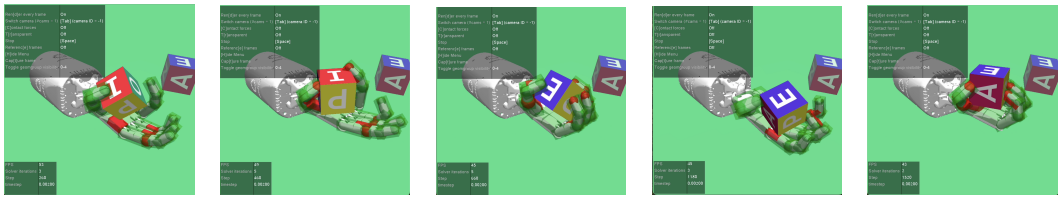
ICA, used in combination with PCA by Berg *et al.* [2] on muscle-activation data, seeks statistically independent latent factors rather than orthogonal, maximally variant ones. For muscle activations, this independence assumption is physiologically motivated, as distinct motor modules are thought to be driven by separate neural commands. For joint-position data, however, it is less natural: the Shadow Hand has independently actuated joints, but their effective commands are strongly coupled by the hand kinematics, contact constraints and task objectives. In this joint space, the covariance structure is precisely the quantity of interest, making variance maximisation a more principled objective than enforcing statistical independence. In addition, ICA lacks a unique closed-form solution and is sensitive to initialisation, which would introduce extra optimisation instability without a clear modelling benefit in the present context. For these reasons, ICA was not pursued further in this thesis.

Given these considerations, PCA offers a pragmatic compromise. It works natively with signed joint-position data in the original dimension, yields a unique orthonormal basis in closed form, and aligns directly with the postural synergy framework of Section 2.2.2, giving the learned components a clear interpretation in terms of kinematic hand synergies. It was therefore a natural starting point for the present experiments. Exploring alternative factorisations such as NMF or ICA for joint-level synergies is an interesting direction for future work.

Chapter 3

Problem Formulation

The background material in Chapter 2 has introduced all the ingredients we will need in order to formulate the problem studied in this thesis: whether constraining the action space to a low-dimensional synergy subspace can accelerate reinforcement learning on novel object manipulation tasks. We begin by describing the simulation environment and the set of objects that define the experimental scope of the thesis. We then formalise the reorientation task as a multi-goal Markov Decision Process (MDP), describe the state, action and goal spaces of the Shadow Dexterous Hand, and motivate the use of sparse binary rewards. Finally, we state the evaluation metrics that will be used throughout the experimental chapter. Every design choice made here will be inherited unchanged by both the full-action-space baseline and the synergy-constrained agent of Chapter 5, so that any difference observed in the experiments can be attributed cleanly to the action-space parameterisation.



(a) **(b)** **(c)** **(d)** **(e)**

Figure 3.1 Sequence of frames from a successful episode on the standard cube reorientation task with touch sensors. The Shadow Dexterous Hand must match the target pose indicated by the semi-transparent object. Frames are sampled at regular intervals and show the progression from the initial configuration (a) to the goal configuration (e).

3.1 Dexterous robotic hands and the simulation environment

The theoretical ingredients of the previous sections take physical form through the specific robotic platform on which the thesis is developed. In this section we briefly describe the Shadow Dexterous Hand and the MuJoCo simulation environment, which together constitute the substrate on which the synergy extraction and reinforcement-

learning pipeline is implemented.

3.1.1 The Shadow Dexterous Hand

Dexterous robotic manipulation requires a mechanical substrate with enough degrees of freedom to approximate the configurations that the human hand can assume. Two design traditions coexist in the field. The first, exemplified by underactuated and soft-synergy hands discussed in Section 2.4, favours mechanical simplicity: the hand has few motors but many passive degrees of freedom coupled by tendons, and it relies on the compliance of the structure to adapt to the object. The second, pursued by the Shadow Robot Company and by the UTah/MIT design lineage, favours a fully actuated anthropomorphic platform: every joint that has a counterpart in the human hand is separately controllable.

This thesis uses a representative member of the second family, the Shadow Dexterous Hand. The Shadow Hand is a five-fingered anthropomorphic robotic hand developed by the Shadow Robot Company; in the version used here it has 24 degrees of freedom, 20 of which are independently actuated while the remaining four are mechanically coupled (the distal interphalangeal joint of each finger, which in the real hand is kinematically linked to the proximal interphalangeal joint). Each actuated joint is driven by a pneumatic or tendon-based actuator that, in the simulation model used in this thesis, is abstracted as an ideal position-controlled servo: the controller issues a target angle, and a low-level loop drives the joint towards it. A labelled rendering of the hand is reported in Figure 3.2.

The Shadow Hand has been adopted in a number of landmark studies on learning-based manipulation, most notably the object reorientation policy of Andrychowicz *et al.* [8] and the multi-goal reorientation benchmarks of Plappert *et al.* [16]. Its combination of high dimensionality, anthropomorphic morphology, and established benchmark status makes it a natural choice for a thesis whose claim concerns the compression of a high-dimensional action space.

3.1.2 MuJoCo and Gymnasium-Robotics

All experiments are conducted in simulation using MuJoCo [25], a general-purpose physics engine designed specifically for robotics and biomechanics. MuJoCo models contact as a convex soft-constraint problem, which is solved at every simulator step by a fast iterative algorithm; this choice makes the simulator numerically stable in the presence of the fast, intermittent contact events that characterise in-hand manipulation, where dozens of finger-object contacts are made and broken every second.

On top of MuJoCo, we use the in-hand manipulation environments shipped with Gymnasium-Robotics [4], an open-source extension of the OpenAI Gym Multi-Goal API originally introduced by Plappert *et al.* The Multi-Goal API enforces that environments expose a goal-aware observation space and a reward function explicitly conditioned on the goal, callable for any alternative goal at training time. This property is essential for the HER relabelling step described in Section 2.1.6.

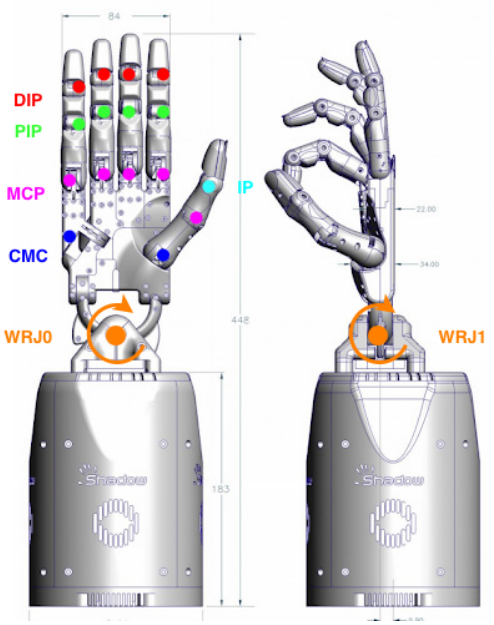


Figure 3.2 The Shadow Dexterous Hand with joint locations annotated. **WRJ0/WRJ1**: Wrist (2 DoF: radial/ulnar deviation and flexion/extension); **MCP**: Metacarpophalangeal joint present on all fingers (2 DoF: horizontal adduction/abduction and vertical flexion/extension); **PIP**: Proximal Interphalangeal joint of the forefinger, middle, ring, and little fingers (1 DoF: flexion/extension); **DIP**: Distal Interphalangeal joint of the forefinger, middle, ring, and little fingers (1 DoF: flexion/extension), not independently actuated, mechanically coupled to the corresponding PIP joint; **CMC**: Carpometacarpal joint of the little finger (1 DoF: abduction) and of the thumb (2 DoF: horizontal and vertical flexion); **IP**: Interphalangeal joint of the thumb (1 DoF: flexion/extension).

Task structure. In every environment the Shadow Hand must reorient a grasped object from a randomised initial pose to a randomised target pose. The task structure follows the benchmark originally introduced by Plappert *et al.* [16]: actions are 20-dimensional absolute joint position commands, the control frequency is 25 Hz, and an episode lasts at most $T = 100$ simulator steps. The reward is sparse and binary, as defined in Section 3.4: the agent receives 0 upon reaching the target pose within a positional tolerance of $\epsilon_p = 1$ cm and a rotational tolerance of $\epsilon_r = 0.1$ rad, and -1 otherwise. In all experiments we use the touch-sensor variant of each environment, in which the observation is augmented with 92 continuous tactile readings distributed across the hand surface. This provides the agent with explicit contact information at every timestep, which is particularly relevant in the synergy-constrained setting where the reduced action space limits the agent's ability to perform fine corrective movements and richer sensory feedback may compensate for this loss of motor resolution.

Object set We evaluate the framework on six manipulation tasks, each defined by a distinct object geometry, listed in Table 3.1. Two of these correspond directly to standard benchmarks from Plappert *et al.*: the standard cube (`HandManipulateBlockRotateXYZ`) and the egg (`HandManipulateEggRotate`), in

which a random target rotation is imposed on all three axes with no target position constraint. The remaining four objects are custom environments implemented by modifying the MuJoCo XML of the standard cube reorientation task, replacing the object geometry while keeping every other aspect of the environment, reward, and observation unchanged. This design ensures that any observed difference in learning performance across objects is attributable solely to the change in object geometry and not to any other environmental factor.

The object set was designed to span a broad range of geometric properties. The three cubes share the same topology but differ in scale, allowing us to test whether synergies transfer across objects that are geometrically identical but physically distinct. The egg is a smooth ellipsoid with no edges or flat faces, substantially altering the contact geometry relative to the cube. The cylinder presents a hollow cylindrical shape with a qualitatively different grasp structure. The sphere is the most symmetric object and the least demanding in terms of contact diversity, providing a natural lower bound on task difficulty.

Table 3.1 Objects used in the experiments and their simulation properties. Dimensions are taken from the MuJoCo XML definition of each environment. For the ellipsoid, r_x, r_y, r_z denote the three semi-axes; for the cylinder, r is the outer radius and h the full height (2× the half-height from the XML). Standard environments are from Plappert *et al.* [16]; custom environments modify the object geometry only.

Object	Role	Type	MuJoCo geom	Dimensions (m)	Origin
Standard cube	Source	Cube	`box`	$0.025 \times 0.025 \times 0.025$	Standard [16]
Small cube	Target	Cube	`box`	$0.020 \times 0.020 \times 0.020$	Custom
Large cube	Target	Cube	`box`	$0.030 \times 0.030 \times 0.030$	Custom
Egg	Target	Ellipsoid	`ellipsoid`	r_x=0.030, r_y=0.030, r_z=0.040	Standard [16]
Cylinder	Target	Cylinder	`cylinder`	r=0.030, h=0.050	Custom
Sphere	Target	Sphere	`sphere`	r=0.033	Custom

3.2 Multi-goal MDP formulation

We model the in-hand reorientation task as a *multi-goal* Markov Decision Process, formalised by Schaul *et al.* [21] through Universal Value Function Approximators (UVFA), which learn a single neural network that approximates $Q(s, a, g)$ across the whole goal space. The tuple is defined as

$$\mathcal{M} \;=\; (\mathcal{S},\ \mathcal{G},\ \mathcal{A},\ P,\ R,\ \gamma), \tag{3.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{G}$ is the goal space, $\mathcal{A}$ is the action space, $P(s' \mid s, a, g)$ is the transition kernel, $R(s, a, g)$ is the goal-conditioned reward, and $\gamma \in [0, 1)$ is the discount factor. At the beginning of each episode a goal $g \in \mathcal{G}$ is sampled from a fixed distribution; the agent is then allowed to interact with the environment for a bounded number of timesteps, receiving at every step a reward that depends jointly on the current transition and on the goal.

The agent is a stochastic policy $\pi(a \mid s, g)$ that conditions on both the current

state and the active goal. Following the notation of Section 2.1.6, its objective is

$$J(\pi) \;=\; \mathbb{E}_{g\sim p(g)}\; \mathbb{E}_{\tau\sim\pi(\cdot|\cdot,g)}\left[\sum_{t=0}^{T-1}\gamma^t\, R(s_t,a_t,g)\right], \tag{3.2}$$

namely the expected discounted return averaged over the goal distribution. The reason to condition on g rather than to learn a separate policy per goal is that the target configuration changes at every episode, and the number of possible goals is uncountable: a single goal-conditioned policy is therefore required to *generalise* across the whole goal space.

3.3 State, action and goal spaces

State space. The observation returned by the simulator at each timestep is a structured dictionary that concatenates (i) the joint positions and joint velocities of the 24 degrees of freedom of the Shadow Dexterous Hand, (ii) the position and orientation of the manipulated object in the hand reference frame, (iii) its linear and angular velocities, (iv) the currently active goal, and (v) 92 continuous tactile readings from the touch sensors distributed across the hand surface. We denote by $s \in \mathcal{S} \subseteq \mathbb{R}^{d_s}$ the vector obtained by flattening this dictionary; for the configuration used throughout the thesis, $d_s = 160$. All joint and pose components are expressed in SI units (radians and metres respectively). Orientations are represented as unit quaternions.

Action space. The baseline (full-action-space) agent acts directly in the *joint command space* of the hand: each action $a \in \mathcal{A}_{\text{full}} \subseteq \mathbb{R}^{20}$ specifies the absolute angular position targets of the 20 independently actuated joints. The remaining four degrees of freedom of the Shadow Hand are mechanically coupled and are therefore not exposed to the controller. Every component of a is bounded between the physical joint limits of the hand and is passed directly to a PD servo that drives the finger towards the commanded target at a fixed internal frequency. The synergy-constrained agent introduced in Chapter 5 will replace this 20-dimensional action space with a much smaller one, while leaving every other element of the MDP unchanged.

Goal space. A goal $g \in \mathcal{G} \subseteq \mathbb{R}^7$ specifies a desired pose of the object in the hand frame, encoded as a 3-dimensional position (in metres) and a 4-dimensional unit quaternion. However, in the `Rotate` variant used throughout this thesis, only the rotational component of the goal varies across episodes: the target position is fixed and the object is expected to remain within the palm throughout the episode. The goal space is therefore effectively $SO(3)$, the group of all orientations in three-dimensional space, sampled uniformly at the beginning of each episode.

3.4 Sparse binary reward

A central and deliberate design choice of the formulation is the adoption of a *sparse binary* reward:

$$R(s,a,g) \;=\; \begin{cases} 0 & \text{if } \|\mathrm{pos}(s)-\mathrm{pos}(g)\| \le \epsilon_p \text{ and } d_{\mathrm{quat}}(\mathrm{rot}(s),\mathrm{rot}(g)) \le \epsilon_r, \\ -1 & \text{otherwise,} \end{cases} \tag{3.3}$$

where $\mathrm{pos}(\cdot)$ and $\mathrm{rot}(\cdot)$ extract the position and orientation of the object, d_{quat} is the standard quaternion distance, and $\epsilon_p = 1\,\mathrm{cm}$ and $\epsilon_r = 0.1\,\mathrm{rad}$ are the positional and rotational tolerances used throughout the thesis.

This choice is unusual in RL for robotics, where it is common to hand-craft a shaped, dense reward that smoothly interpolates between "far from goal" and "at the goal". We deliberately avoid reward shaping for two reasons. First, designing a shaped reward for dexterous manipulation is hard: it requires assumptions about what a good intermediate behaviour should look like, and these assumptions can easily bias the policy towards visually plausible but suboptimal strategies [33]. Second, and more importantly, we want the learning signal to depend as little as possible on human intuition, so that the only differences between two trained agents are the ones induced by the representation of the action space, not by the reward engineer. The price to pay for this cleanliness is that an episode rarely produces a non-trivial gradient by chance: as discussed in Section 2.1.6, this motivates the use of Hindsight Experience Replay, which recovers a learning signal from otherwise uninformative trajectories.

3.5 Evaluation metrics

We evaluate trained agents along three complementary axes, each corresponding to a distinct question about the quality of the learned policy.

Success rate. The primary metric is the fraction of evaluation episodes in which the object is reoriented within the tolerances (ϵ_p, ϵ_r) before the episode terminates. Unless otherwise stated, success rates are estimated over 200 held-out episodes with freshly sampled goals, and are reported at regular intervals during training so as to produce learning curves.

Sampling duration. To quantify *how fast* a policy is learned, we report the number of environment timesteps required to reach a prescribed success threshold, typically 80%. A lower sampling duration means that the synergy-constrained representation has accelerated learning, requiring fewer interactions with the environment before a policy of sufficient quality is learned.

Wall-clock time. The wall-clock time is the total elapsed time in hours and minutes required for the training process to reach the success threshold, measured on a single fixed GPU configuration described in Section 5.6. Unlike the sampling duration, it also captures the per-step computational cost of the network updates, and

is more immediately interpretable for readers who are not familiar with reinforcement learning environments.

For the success rate, we report the mean and standard deviation across seeds as a shaded learning curve. For the sampling duration and wall-clock time, we report the mean across seeds as a bar chart, with individual seed values shown as overlaid dots.

Chapter 4

Related Work

This chapter surveys the body of work most closely related to the present thesis. The literature spans three overlapping research threads: the use of motor synergies for dexterous robotic manipulation, the application of reinforcement learning to in-hand manipulation tasks, and the question of generalisation and transfer across objects and tasks. We conclude the chapter by explicitly positioning our contribution with respect to the reviewed works.

4.1 Motor synergies in robotic grasping and manipulation

The neuroscientific observation that the human hand operates through a low-dimensional set of coordinated joint patterns, known as motor synergies, was first formalised in a robotic context by Santello, Flanders and Soechting [34], who showed that five principal components suffice to reconstruct the full repertoire of human grasping postures with high fidelity. This seminal result inspired a long line of work on synergy-based robot hand control.

Ciocarlie and Allen were the first to exploit postural synergies for autonomous grasp planning [28]. By projecting candidate hand configurations onto the first two principal synergies of Santello et al., they reduced what is otherwise a high-dimensional search in configuration space to a two-dimensional problem, obtaining high-quality grasps with negligible loss in success rate. Their work established the core computational argument that a low-dimensional synergy basis is not merely a compact description of human data but a practically useful prior for robotic search.

Bicchi and collaborators extended this idea to mechanical design through the theory of soft synergies and synergy-based underactuation [26]. Rather than imposing synergies in software, they embedded them into the physical structure of the hand through compliant tendon-pulley couplings. The Pisa/IIT SoftHand [22] is the most prominent result of this line: a single-motor hand capable of grasping a wide variety of objects by virtue of its mechanically realised first synergy.

Hayashibe and Shimoda introduced the synergetic learning control paradigm, in which a low-dimensional synergy basis is treated as a redundancy-resolution device that coordinates the joints of an over-actuated robot through a small number of virtual command channels [15]. This work provided an explicit algorithmic framework

for using synergies not only as a description tool but as an active component of a closed-loop controller, and served as a direct precursor to the learning-based approaches surveyed below.

4.2 Learning motor synergies from reinforcement learning policies

The approaches described in Section 4.1 all rely on synergies measured on human subjects or designed by hand. A different question is whether a synergy basis can be *discovered automatically* from the behaviour of a learning agent, and whether such a data-driven basis can be reused across tasks.

Chai and Hayashibe [9] were the first to ask whether high-performing deep RL policies spontaneously develop motor synergies. Analysing the joint trajectories produced by agents trained with policy-gradient methods, they found that the degree of synergy structure in the learned trajectories correlates with task performance, providing the first explicit bridge between the synergy hypothesis of motor neuroscience and modern deep RL.

The work most directly related to the present thesis is that of Kutsuzawa and Hayashibe [5]. They trained a PPO agent on a planar reaching task, extracted spatio-temporal synergies from the learned joint-torque trajectories via non-negative matrix factorisation (NMF), and showed that combining the synergy repertoires obtained from two single-plane policies yields a basis whose span covers entirely new planes that neither single repertoire could reach alone. This is the first demonstration that synergies extracted by an RL agent support *compositional generalisation* to novel targets, not merely compression of the training data. Our work inherits this core insight and extends it to the harder setting of multi-finger in-hand manipulation with sparse rewards, replacing NMF with PCA and PPO with SAC combined with HER.

At the muscle level, Berg, Caggiano and Kumar [2] developed the Synergistic Action Representation (SAR), obtained by composing Independent Component Analysis and PCA on the muscle activations produced by a play-phase policy on simple tasks with simulated Shadow Hand. The resulting basis generalises zero-shot to in-hand reorientation of a hundred novel objects and to locomotion on varied terrains, while end-to-end RL baselines with the same sample budget fail on both. Our work pursues a similar hypothesis but at the kinematic rather than muscle level, and within a SAC+HER training loop rather than a zero-shot transfer regime.

4.3 Reinforcement learning for dexterous in-hand manipulation

The use of RL for dexterous manipulation gained prominence with the work of Andrychowicz et al. [8], who trained a policy that performs vision-based object reorientation on a physical Shadow Dexterous Hand using domain randomisation, demonstrating that policies trained entirely in simulation can transfer to a real robot. While impressive in scale, this work operated in the full 20-dimensional joint space

and required enormous computational resources, highlighting the sample inefficiency that motivates the synergy-constrained approach of the present thesis.

Plappert et al. [16] introduced the multi-goal manipulation benchmarks built on the Shadow Hand, which are the environments used throughout this thesis. They also provided the first systematic evaluation of DDPG combined with HER on these benchmarks, establishing the baseline performance figures against which our results are compared.

Hindsight Experience Replay was introduced by Andrychowicz et al. [18] to address the fundamental difficulty of learning from sparse binary rewards in high-dimensional spaces. By relabelling failed trajectories with goals that were actually achieved, HER recovers a dense learning signal from episodes that would otherwise carry no gradient. HER is essential for our setting, as discussed in Chapter 2.

For policy optimisation, we adopt Soft Actor-Critic (SAC) [13], an off-policy maximum-entropy algorithm that has become the standard for continuous-control benchmarks due to its combination of sample efficiency, stable optimisation, and principled exploration.

4.4 Action space reduction for efficient learning

Beyond synergies, several works have investigated action space reduction as a strategy for improving RL efficiency in high-dimensional control problems. Al Borno, Hicks and Delp [7] showed, in a musculoskeletal upper-extremity model, that a small repertoire of motor modules suffices to drive a wide range of reaching tasks and improves both learning speed and generalisation to new targets. Their work is one of the cleanest computational demonstrations that extracted synergies are not merely a redescription of training data but genuinely useful priors for learning new tasks.

Rückert and d'Avella [24] implemented a shared-synergy structure based on dynamic movement primitives and showed that the resulting low-dimensional representation accelerates learning in a simulated musculoskeletal system. The shared nature of the basis, namely that the same primitives are reused across multiple tasks, is the key feature that enables transfer, and is precisely the property exploited in Stage C of our framework.

4.5 Positioning of this work

Table 4.1 summarises the key dimensions along which the present thesis differs from the most closely related works.

The present thesis occupies a specific and previously unexplored position in this space. Like Santello et al. and Ciocarlie and Allen, we study postural-level kinematic synergies extracted via PCA. Following the line of work developed by Hayashibe and collaborators [5, 9], we learn these synergies from the trajectories of a reinforcement-learning agent rather than from human data. In the same spirit as Berg et al. [2], we demonstrate that a fixed synergy basis can be reused on novel tasks without retraining the synergy model.

Unlike any of the above, we combine these ingredients in the specific setting of the Shadow Dexterous Hand solving a multi-goal in-hand reorientation task with

Table 4.1 Comparison of closely related works. All platforms are simulation-based unless marked (real). *Transfer* indicates whether the synergy basis is reused across tasks without retraining the synergy model.

Work	Synergy type	Platform	RL algorithm	Transfer
Ciocarlie & Allen [28]	Kinematic (human)	Barrett Hand (real)	N/A	No
Chai & Hayashibe [9]	Kinematic	Arm	TD3/SAC	No
Kutsuzawa & Hayashibe [5]	Spatio-temporal	Arm	PPO	Yes
Berg et al. [2]	Muscle	Hand & leg	SAC	Yes
This work	**Kinematic**	**Shadow Hand**	**SAC+HER**	**Yes**

sparse binary rewards, using SAC combined with HER. This combination introduces challenges absent from prior work: the goal-conditioned observation space required by HER, the multi-finger contact dynamics of the Shadow Hand, and the need to show that a kinematic synergy basis extracted from a single source object accelerates learning and maintains performance on objects whose geometry was never seen during synergy extraction.

Chapter 5

Proposed Method

This chapter describes the *Motor Synergy Generalization Framework*, the central contribution of this thesis. The framework is a three-stage pipeline. Stage A trains a full-DoF SAC+HER agent on a single *source* task until convergence. Stage B extracts a set of spatial synergies offline from the joint-action trajectories produced by the converged Stage A policy via Principal Component Analysis. Stage C uses these synergies as a fixed linear decoder and trains new RL agents to act in the resulting low-dimensional latent space, for each of the novel *target* tasks. Crucially, the synergy basis is frozen after Stage B: it is never re-learned, re-fitted or fine-tuned on the target tasks. This design choice is what turns the pipeline from a simple dimensionality-reduction trick into a proper *generalisation* framework, and it is the element whose empirical validity the experiments in Chapter 6 are designed to test.

5.1 Overview of the three-stage framework

The overall pipeline is illustrated in Figure 5.1. In Stage A, a reinforcement-learning agent with full access to the 20-dimensional joint command space is trained on a chosen source task until convergence using SAC combined with HER. In Stage B, the learned policy is frozen and executed for a fixed number of evaluation rollouts; the 20-dimensional joint commands are collected into a data matrix, and Principal Component Analysis is applied to extract a small set of spatial *synergies*, each representing a coordinated activation pattern of the actuated joints. These synergies form the rows of a fixed matrix $W \in \mathbb{R}^{K\times 20}$ that defines a linear map from a K-dimensional latent space to the full joint space, together with a mean action vector $\bar{x}$.

In Stage C, a new agent is trained on a target task. The policy no longer outputs joint commands: instead, it outputs a K-dimensional vector of synergy activations, which is decoded through the fixed W at every timestep and then sent to the simulator as a full 20-dimensional joint command. From the perspective of the underlying simulator, nothing has changed; from the perspective of the learning algorithm, the effective action space dimension has decreased from 20 dimensions to K. We will set $K = 5$ by default, motivated by the analysis of Section 5.3.3.

The separation between the three stages embodies a strong claim about motor control, namely that a small number of linear coordination patterns, extracted from

one task, carry enough structure to speed up learning on a family of related tasks. Sections 5.2, 5.3 and 5.4 describe each stage in detail.

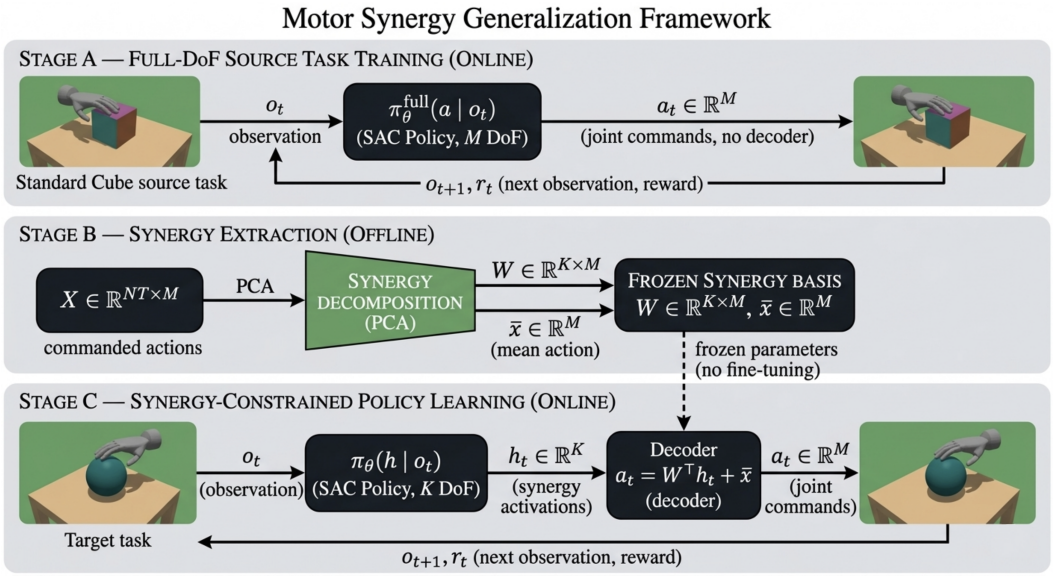


Figure 5.1 Overview of the Motor Synergy Generalization Framework. Stage A trains a full-DoF SAC agent on the source task. Stage B extracts the synergy basis offline via PCA. Stage C trains a synergy-constrained SAC agent on each target task using the frozen decoder.

5.2 Stage A: Full-DoF source task training

5.2.1 Source task selection

The source task determines the synergy basis that every target agent will operate in. Its choice must therefore satisfy two competing requirements: it must be rich enough that the policy solving it exercises a wide portion of the hand's coordination repertoire, so that the extracted synergies span directions of genuine manipulation interest; and a full-DoF SAC+HER agent must reach a stable, high-performing policy on it, so that the rollouts collected in Section 5.3.1 reflect genuine coordinated manipulation behaviour rather than undirected exploration.

We select the *standard cube* task, the reorientation of a cube of side 0.025 m as defined in Table 3.1, as the source task. The cube is an object with flat faces, sharp edges, and no rotational symmetry, which means that achieving an arbitrary target orientation requires the hand to produce a rich and varied repertoire of multi-finger coordination patterns: different faces must be brought forward, the object must be rolled, tilted and stabilised, and the fingers must continuously readjust their contacts to prevent the cube from dropping. This diversity of required coordination is precisely what makes the cube task a good source of synergies: the joint trajectories produced by a converged policy span many directions in the 20-dimensional action space, and a PCA decomposition of those trajectories is likely to capture coordination patterns that are relevant beyond the specific geometry of the cube.

The choice is further supported by the established role of the cube task as a standard benchmark in the dexterous manipulation literature [8, 16], which gives us confidence that a converged policy represents genuine dexterous behaviour rather than an artefact of the simulation. The Stage A agent is trained for 6×10^6 environment timesteps, deliberately longer than the 3×10^6 timesteps used for the target agents in Stage C. Although our experiments show that meaningful performance is already achieved at 3×10^6 timesteps, the additional training ensures that the policy has fully converged and that the collected rollouts reflect the hand's mature coordination repertoire rather than an intermediate learning stage. The quality of the synergy basis depends directly on the quality of the trajectories from which it is extracted, and we therefore prioritise convergence over training speed at this stage.

5.3 Stage B: Synergy extraction

5.3.1 Data collection protocol

Once the Stage A agent has converged, we freeze the policy (no further gradient updates) and execute it for $N = 200$ evaluation rollouts of fixed length $T = 100$ timesteps each. At every timestep of every rollout we record the commanded joint action, which is a vector in $\mathbb{R}^{20}$. The resulting data tensor has shape $N \times T \times M = 200 \times 100 \times 20$; we reshape it into a matrix

$$X \in \mathbb{R}^{NT \times M} = \mathbb{R}^{20{,}000 \times 20}, \tag{5.1}$$

where each row corresponds to a single 20-dimensional command issued by the policy at some timestep of some rollout. X is the raw material from which synergies are extracted.

Two design decisions are worth explicit mention. First, we record the *commanded* joint actions rather than the realised joint positions, because our goal is to capture the coordination patterns that are used by the controller, not those that the hand's low-level dynamics impose. Second, we use $N = 200$ rollouts to ensure that the covariance structure of X reflects the full diversity of goals and initial conditions encountered during manipulation.

5.3.2 PCA-based synergy extraction

Motor synergies are extracted from X via Principal Component Analysis, following the procedure described in Section 2.5. PCA is applied to the centred action matrix $\tilde{X}$ and yields an orthonormal matrix $W \in \mathbb{R}^{K \times 20}$ whose rows are the top-K eigenvectors of the empirical covariance matrix, ordered by decreasing eigenvalue. Each row $w_k \in \mathbb{R}^{20}$ is a *spatial synergy*: a coordinated pattern of joint activations that can be linearly combined with other synergies to approximate the original joint commands. The action matrix is thus factorised as

$$X \approx \bar{x} + H\,W, \qquad H \in \mathbb{R}^{NT \times K}, \tag{5.2}$$

where $\bar{x}$ is the mean action vector and the rows of H are the corresponding *synergy activations*. In Stage C the roles of H and W are reversed: W is frozen and the

policy learns to produce activations $h_t \in \mathbb{R}^K$ that are decoded through W into full 20-dimensional joint commands at every control step.

5.3.3 Choice of the number of synergies

The hyperparameter K controls the fundamental trade-off of the method: a large K preserves almost all the information in X but barely reduces the effective action space, while a small K aggressively compresses the action space at the risk of discarding motor patterns that the target tasks would actually need.

To guide this choice, we evaluate reconstruction quality using the coefficient of determination $R^2(K)$ defined in equation (2.17). Rather than computing a single global R^2, we compute it separately for each of the $M = 20$ joint dimensions and report the mean across dimensions:

$$\overline{R^2}(K) \;=\; \frac{1}{M}\sum_{j=1}^{M} R_j^2(K), \tag{5.3}$$

where $R_j^2(K)$ is the coefficient of determination for joint j using K synergies. This per-DoF averaging is more informative than a global scalar because it detects cases where a small number of joints are poorly reconstructed even when the global variance is well-explained.

Reconstruction quality is evaluated on a held-out test set obtained by splitting the $N = 200$ rollouts into a training fold (80%) and a test fold (20%), where the split is performed *by trajectory* rather than by timestep. Splitting by timestep would allow temporally correlated frames from the same episode to appear in both folds, inflating the reported R^2; splitting by trajectory ensures that the test episodes are entirely unseen during synergy extraction.

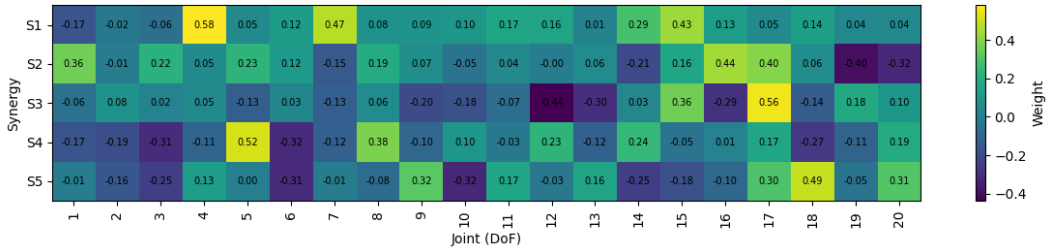
Figure 5.2 (b) shows $\overline{R^2}(K)$ as a function of K: reconstruction quality rises steeply in the first few components and saturates only slowly after, with $K \approx 17$ required to exceed $\overline{R^2} > 0.95$.

Saturating $\overline{R^2}$ is however not the correct objective: retaining 17 of 20 dimensions leaves the action space almost as large as the original, which would defeat the purpose of the method. We therefore retain only the first $K = 5$ synergies. This setting captures approximately 50% of the total variance, a reconstruction that would be mediocre in most statistical contexts but is, as we shall see in Chapter 6, entirely sufficient to learn the target tasks.

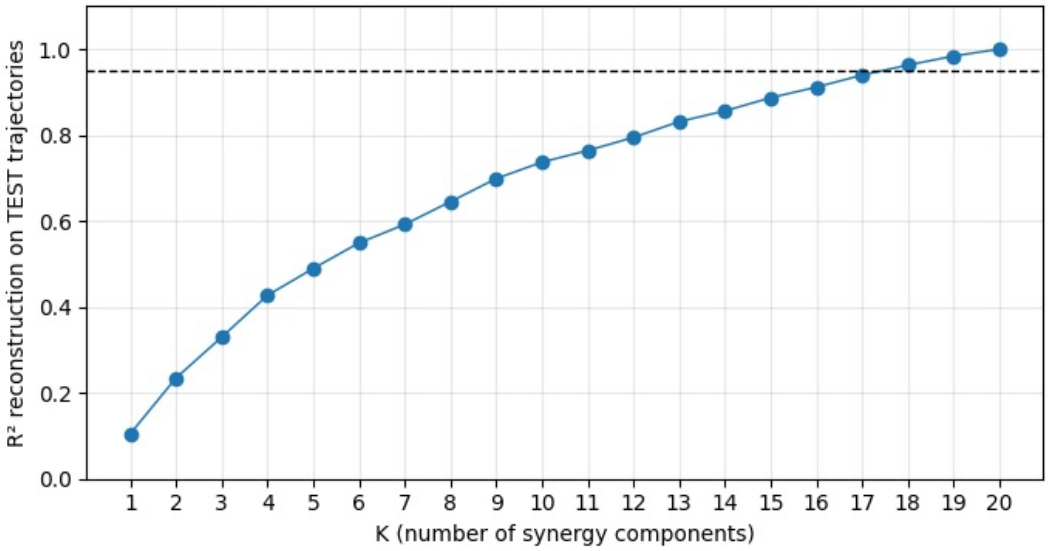
5.4 Stage C: Synergy-constrained policy learning

5.4.1 Low-dimensional action space

In Stage C the agent acts in a new, K-dimensional action space $\mathcal{A}_{\text{syn}} \subseteq \mathbb{R}^K$, whose elements we denote by $h \in \mathbb{R}^K$ and interpret as synergy activations. The K-dimensional action space is enforced by the `HandSynergyEnv` wrapper, which replaces the native 20-dimensional action space of the base environment with a K-dimensional `Box` space bounded by $[-h_{\max}, h_{\max}]$. The SAC policy therefore only ever observes and produces K-dimensional vectors. The decoding to full joint



(a) Heatmap of the first five extracted synergies $W \in \mathbb{R}^{5\times 20}$. Rows correspond to synergy components, columns to joint DoF. Colour intensity indicates the contribution of each joint to the corresponding synergy.



(b) Mean per-DoF reconstruction quality $\overline{R^2}$ as a function of the number of retained synergy components K, evaluated on held-out trajectories. The dashed line marks $\overline{R^2} = 0.95$.

Figure 5.2 Motor synergy extraction from cube manipulation data using PCA.

commands happens inside the wrapper's `step` method via equation (5.4), and the resulting 20-dimensional commands are clipped to the simulator's native bounds $[-1, 1]$ before being applied to the hand.

The choice of $h_{\max}$ has a direct effect on learning performance. Setting $h_{\max} = 1.0$, which naively matches the native action bounds, severely limits the expressiveness of the synergy-constrained policy: because the PCA activations extracted from the source task data span a range considerably larger than $[-1, 1]$, constraining the policy to this interval prevents it from reproducing the full range of joint configurations observed during Stage B. In our experiments this resulted in poor learning progress across all target tasks. Expanding the range to $h_{\max} = 3.0$ resolved this issue and produced a significant improvement in learning performance across all objects. We therefore set $h_{\max} = 3.0$ throughout all experiments.

By construction, $\mathcal{A}_{\text{syn}}$ is a strict subset of the original joint space $\mathcal{A}_{\text{full}}$. Some motor behaviours that were attainable in the baseline are therefore no longer attainable in the synergy-constrained setting. This is the central structural hypothesis

of the method: we conjecture that the subspace spanned by a handful of dominant synergies, extracted on a sufficiently rich source task, already contains the motor patterns needed to solve a variety of target tasks on the same hand.

5.4.2 Fixed linear decoder

The map from synergy activations to joint commands is the *fixed* affine decoder

$$a_t \;=\; W^{\top} h_t \;+\; \bar{x}, \tag{5.4}$$

where $W \in \mathbb{R}^{K \times M}$ and $\bar{x} \in \mathbb{R}^{M}$ are the synergy matrix and the mean command extracted in Stage B, and $h_t \in \mathbb{R}^{K}$ is the synergy activation produced by the policy at time t. Equation (5.4) is evaluated by a single matrix-vector product at every control step; its computational cost is negligible.

The word *fixed* is essential. W and $\bar{x}$ are not trainable parameters of the Stage C agent: they do not receive gradient updates during policy optimisation, nor are they re-fitted when switching from one target task to another. A single synergy basis, extracted in Stage B from the cube task of Section 5.2.1, is reused identically for every target object. This is what allows the framework to be interpreted as a *generalisation* mechanism rather than as a per-task compression technique: the claim tested by the experiments is not merely that low-dimensional action spaces accelerate learning, but that a low-dimensional action space *obtained from one task* accelerates learning on other tasks.

5.4.3 Policy architecture

The policy $\pi_\theta(h \mid s, g)$ and the two critics $Q_{\phi_1}, Q_{\phi_2}(s, g, h)$ are multi-layer perceptrons with two hidden layers of 256 units and ReLU activations, matching the default architecture of SAC [13]. The actor outputs the mean and log-standard-deviation of a Gaussian distribution over synergy activations; a hyperbolic tangent squashing function is applied to the sampled action to enforce the bounds of $\mathcal{A}_{\text{syn}}$. The critics take as input the concatenation of state, goal and synergy activation, and output a scalar.

The only architectural difference between the full-action-space baseline and the synergy-constrained agent is the output dimension of the actor and the corresponding input dimension of the critic: 20 in the baseline and $K = 5$ in the synergy-constrained agent. All other hyperparameters are kept identical so that any observed difference in learning performance between the two agents can be attributed solely to the action-space parameterisation, as required by the experimental design of Chapter 3.

5.5 Training procedure

We train the Stage C agents with SAC combined with HER in its *future* strategy, as described in Sections 2.1.5 and 2.1.6. The full training loop is given in Algorithm 1. At each environment step the policy samples a synergy activation h_t, which is decoded into a joint command via equation (5.4), applied to the hand, and stored in the replay buffer together with the active goal g. At every gradient step, a mini-batch is sampled from the replay buffer and HER relabelling is applied: the stored goal

Algorithm 1 Synergy-Constrained SAC+HER (Stage C)

Require: Fixed synergy matrix W and mean $\bar{x}$ from Stage B; initial parameters θ, ϕ_1, ϕ_2; target parameters $\bar{\phi}_1, \bar{\phi}_2$; replay buffer $\mathcal{D} = \emptyset$.

for episode $= 1, \ldots, E$ **do**
  Sample initial state s_0 and goal g from the environment.
  for $t = 0, \ldots, T-1$ **do**
    Sample synergy activation $h_t \sim \pi_\theta(\cdot \mid s_t, g)$.
    Compute joint command $a_t = W^\top h_t + \bar{x}$. ▷ frozen decoder
    Execute a_t, observe s_{t+1} and reward $r_t = R(s_t, a_t, g)$.
    Store $(s_t, h_t, r_t, s_{t+1}, g)$ in $\mathcal{D}$.
  end for
  for each gradient step **do**
    Sample a mini-batch $\mathcal{B} \subset \mathcal{D}$.
    Replace each g in $\mathcal{B}$ by a future-strategy hindsight goal g' and recompute $r = R(s, a, g')$.
    Update ϕ_i by minimising the soft Bellman loss on $\mathcal{B}$, for $i = 1, 2$.
    Update θ by the reparameterised SAC policy gradient.
    Update the temperature α to track the target entropy.
    Soft-update the targets: $\bar{\phi}_i \leftarrow \tau\phi_i + (1-\tau)\bar{\phi}_i$.
  end for
end for

g is replaced with a hindsight goal g' sampled from the future states of the same episode, and the reward is recomputed using the environment reward function. The relabelled batch is used to update the critics and the actor exactly as in standard SAC. The synergy decoder W is never updated.

5.6 Implementation details

The framework is implemented in Python, building on PyTorch [11] for the neural networks and on Stable-Baselines3 [6] for the SAC+HER training loop. The simulation environments are provided by Gymnasium-Robotics [4], running on top of MuJoCo [25]. PCA is performed with scikit-learn [27]; the estimated synergy matrix W is stored as a `.npz` file and loaded, unchanged, at the beginning of every Stage C run.

The hyperparameters used throughout all experiments are summarised in Table 5.1.

Hyperparameter	Value
Optimiser	Adam
Learning rate (actor and critics)	3×10^{-4}
Batch size	256
Discount factor γ	0.99
Target smoothing coefficient τ	5×10^{-3}
Replay buffer size	10^6
Learning starts	10,000
Gradient steps per environment step	1
HER strategy	future
HER relabelled goals per transition	8
Entropy target	$-0.8 \times \dim(\mathcal{A})$
Synergy activation range $h_{\max}$	3.0
Observation normalisation	VecNormalize, clip = 10.0
Hidden layers (actor and critics)	$[256, 256]$
Activation	ReLU
Training timesteps	3×10^6

Table 5.1 SAC+HER hyperparameters shared by the baseline and the synergy-constrained agents.

The only quantity that differs between the baseline and the synergy-constrained agent is the output dimension of the actor: 20 for the baseline and $K = 5$ for the synergy-constrained agent. Each target-task run lasts for 3×10^6 environment timesteps, corresponding to a wall-clock time of approximately 8 hours for the baseline and 5 to 6 hours for the synergy-constrained agent depending on the object. Every experiment is repeated with multiple random seeds, and all learning curves in Chapter 6 report the mean and standard deviation over seeds.

Chapter 6

Experimental Results

This chapter presents the experimental evaluation of the Motor Synergy Generalization Framework introduced in Chapter 5. The central question is whether a synergy basis extracted from a single source task accelerates policy learning on a set of novel object manipulation tasks, and whether this advantage holds consistently across objects with substantially different geometries. We compare two agents on each target task: a full-action-space baseline that operates directly in the 20-dimensional joint space, and a synergy-constrained agent that operates in the 5-dimensional synergy activation space defined in Section 5.4.1. All other aspects of the training setup are identical between the two agents, so that any observed difference in learning performance can be attributed solely to the action-space parameterisation. The five target objects used for this evaluation are shown in Figure 6.1.

(a) Small Cube **(b)** Large Cube **(c)** Egg **(d)** Cylinder **(e)** Sphere

Figure 6.1 The five target objects used for the experimental evaluation in the MuJoCo simulator.

6.1 Experimental setup

The experimental setup follows directly from the problem formulation of Chapter 3 and the implementation details of Chapter 5. We briefly summarise the key elements here for clarity.

Agents. For each target task we train two agents. The *full-action-space* agent operates directly in the native 20-dimensional joint command space of the Shadow Dexterous Hand. The *synergy-constrained* agent operates in the 5-dimensional synergy activation space, with actions decoded at every control step through the fixed linear decoder of equation (5.4). The synergy basis W is extracted once from

the standard cube task as described in Section 5.3 and reused without modification across all target tasks.

Target tasks. We evaluate both agents on five target tasks corresponding to the five non-source objects defined in Table 3.1: the small cube, the large cube, the egg, the cylinder, and the sphere. Each task uses the touch-sensor variant of the corresponding environment, in which the observation is augmented with 92 continuous tactile readings distributed across the hand surface.

Training protocol. Each agent is trained for 3×10^6 environment timesteps using SAC combined with HER, with the hyperparameters listed in Table 5.1. Every experiment is repeated with 3 independent random seeds. For the full action space agent on both cube tasks, one seed was excluded due to a sudden numerical instability early in training; the corresponding metrics are therefore computed over two seeds. The remaining seeds show consistent behaviour and the exclusion does not bias the evaluation.

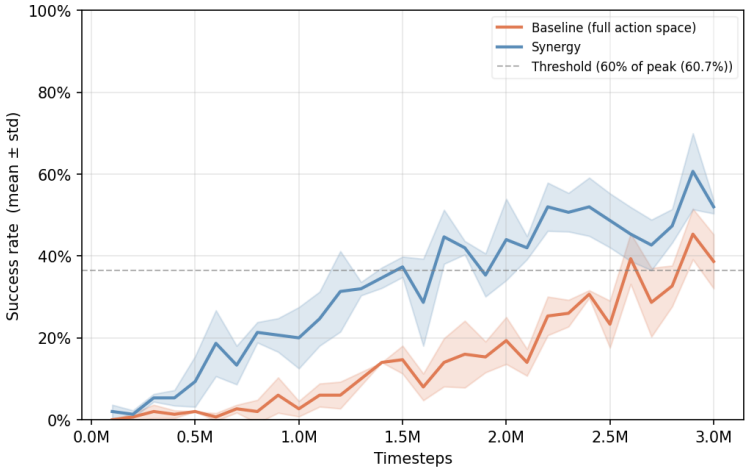
Evaluation. At regular intervals during training, the current policy is evaluated over 50 deterministic rollouts with freshly sampled goals. The primary metric is the success rate, defined as the fraction of evaluation episodes in which the object is reoriented within the tolerances (ϵ_p, ϵ_r) before the episode terminates, as defined in Section 3.4. We report the success rate as a function of environment timesteps, together with the sampling duration and wall-clock time required to reach the success threshold.

6.2 Results on target tasks

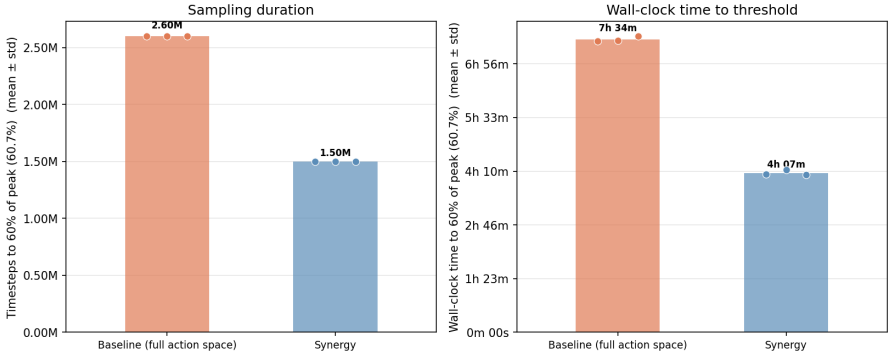
6.2.1 Small cube

The small cube has a side of 0.020 m, smaller than the standard cube from which the synergies were extracted. Neither agent reaches the absolute 80% success threshold within the 3×10^6-timestep budget; the peak mean success rate across both agents is 60.7%, so we adopt a relative threshold of 36.4% (60% of the peak) for the comparisons. The learning curves are shown in Figure 6.2.

The synergy-constrained agent shows the strongest advantage among the cube variants on this task. It reaches the 36.4% threshold in 1.50×10^6 timesteps (4 h 07 m), compared to 2.60×10^6 timesteps (7 h 34 m) for the baseline: a reduction of approximately 42% in sampling duration and 46% in wall-clock time. We attribute this to the geometric proximity between the small cube and the standard source cube: although the object is scaled down, the cubic topology is identical. The synergy subspace therefore provides an accurate and useful prior for exploration from the very beginning of training, explaining both the earlier onset of learning and the higher final performance relative to the large cube task.



(a) Success rate over training.



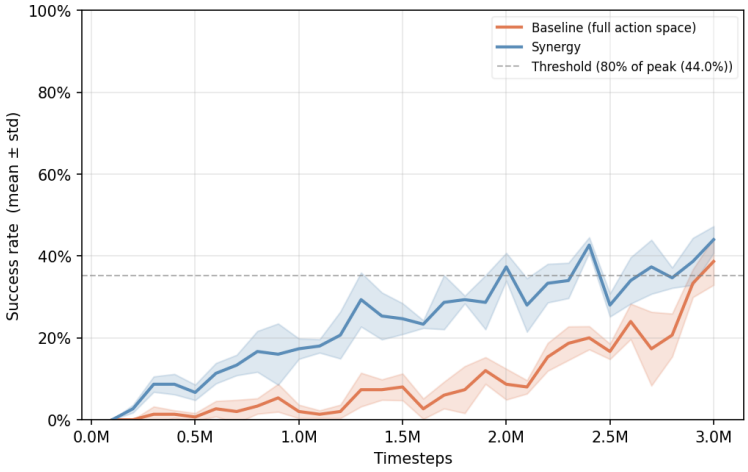
(b) Sampling duration and wall-clock time to threshold.

Figure 6.2 Small cube case. Comparison of baseline (full action space) and synergy-reduced SAC+HER, across 3 seeds: (a) success rate over training; (b) sampling duration and wall-clock time to threshold.

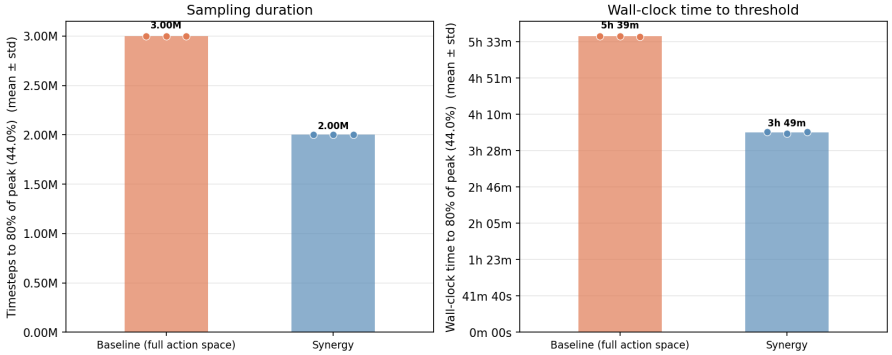
6.2.2 Large cube

The large cube has a side of 0.030 m, larger than the standard cube from which the synergies were extracted. Neither agent reaches the absolute 80% success threshold within the 3×10^6-timestep budget; the peak mean success rate across both agents is 44.0%, noticeably lower than the 60.7% achieved on the small cube, confirming that the larger cube is the more demanding of the two cube variants. We adopt a relative threshold of 35.2% (80% of the peak) for the comparisons. The learning curves are shown in Figure 6.3.

The synergy-constrained agent reaches the 35.2% threshold in 2.00×10^6 timesteps (3 h 49 m), compared to 3.00×10^6 timesteps (5 h 39 m) for the baseline: a reduction of approximately 33% in sampling duration and 32% in wall-clock time. While the synergy advantage is smaller than on the small cube, it remains consistent: the cubic topology is identical to the source task, and the synergy subspace continues to provide a useful exploration prior despite the increased object size.



(a) Success rate over training.



(b) Sampling duration and wall-clock time to threshold.

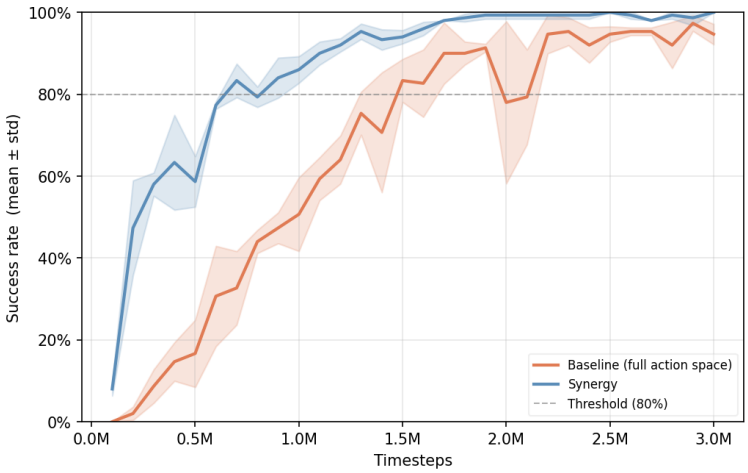
Figure 6.3 Large cube case. Comparison of baseline (full action space) and synergy-reduced SAC+HER, across 3 seeds: (a) success rate over training; (b) sampling duration and wall-clock time to threshold.

6.2.3 Egg

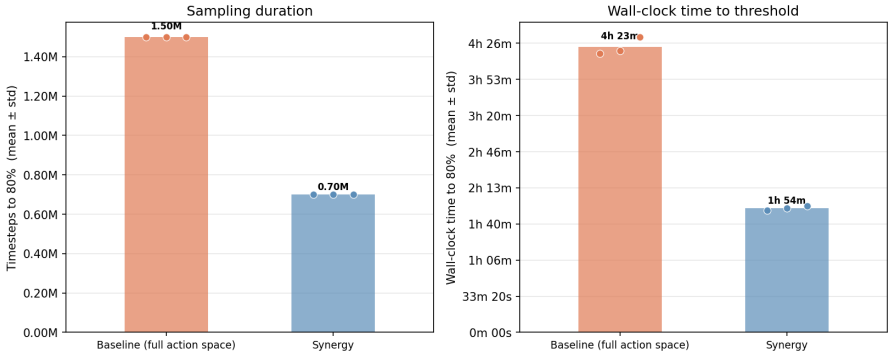
The egg is a prolate ellipsoid with semi-axes $r_x = r_y = 0.030\,\text{m}$ and $r_z = 0.040\,\text{m}$, referred to hereafter as the *egg* for brevity. Unlike the cube, it has no edges or flat faces, which fundamentally changes the contact geometry: the fingers must exert carefully balanced normal forces on a curved surface to prevent the object from rolling out of the grasp. Both agents ultimately converge to near-perfect performance within the training budget, confirming that the egg is a substantially more tractable task than either cube variant. Both agents reach the absolute 80% success threshold, so we use that level for the comparison. The learning curves are shown in Figure 6.4.

The synergy-constrained agent reaches the 80% threshold in 0.70×10^6 timesteps (1 h 54 m), compared to 1.50×10^6 timesteps (4 h 23 m) for the baseline — a reduction of approximately 53% in sampling duration and 57% in wall-clock time. This result is particularly notable because the egg geometry is qualitatively different from the cubic source task: the synergy basis was extracted exclusively from cube manipulation trajectories, yet it transfers effectively to an object with a smooth curved surface

and no edges.



(a) Success rate over training.



(b) Sampling duration and wall-clock time to threshold.

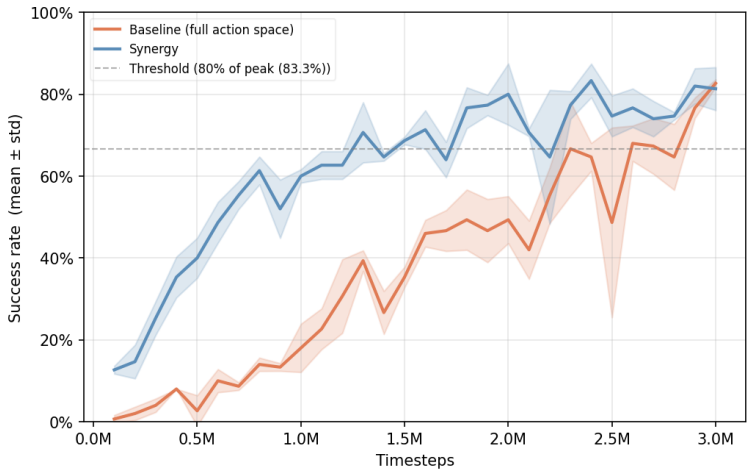
Figure 6.4 Egg case. Comparison of baseline (full action space) and synergy-reduced SAC+HER, across 3 seeds: (a) success rate over training; (b) sampling duration and wall-clock time to threshold.

6.2.4 Cylinder

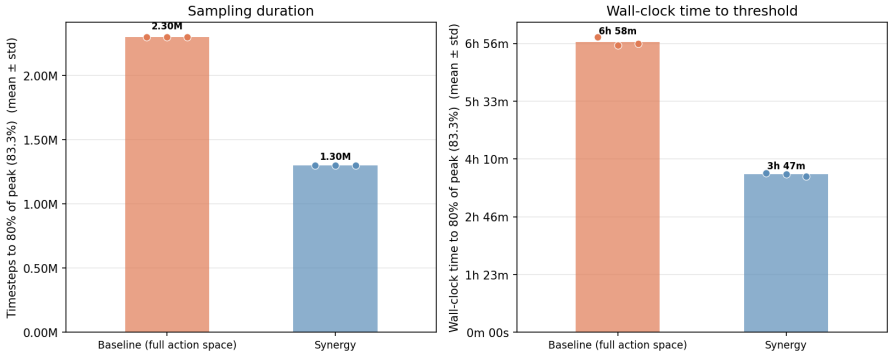
The cylinder task reaches a peak mean success rate of 83.3% across both agents, just below the absolute 80% threshold for a reliable comparison. We therefore adopt a relative threshold of 66.7% (80% of the peak) for the analysis. The learning curves are shown in Figure 6.5.

The synergy-constrained agent reaches the 66.7% threshold in 1.30×10^6 timesteps (3 h 47 m), compared to 2.30×10^6 timesteps (6 h 58 m) for the baseline — a reduction of approximately 43% in sampling duration and 46% in wall-clock time. The cylinder presents a distinctive grasping challenge: unlike the cube, it has no edges to anchor the fingers against, and its flat end-caps alternate with a curved lateral surface, requiring the hand to adapt its contact configuration depending on the current orientation of the object. Despite this added complexity, the synergy basis transfers effectively, suggesting that the low-dimensional coordination patterns it encodes are

sufficiently general to handle objects whose geometry differs substantially from the source cube.



(a) Success rate over training.



(b) Sampling duration and wall-clock time to threshold.

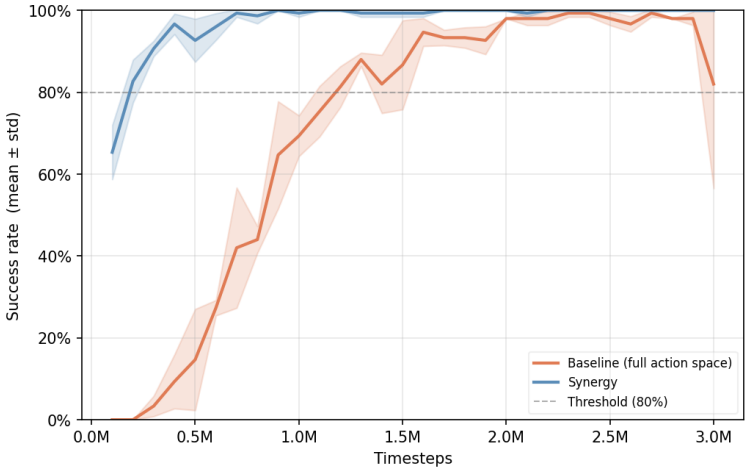
Figure 6.5 Cylinder case. Comparison of baseline (full action space) and synergy-reduced SAC+HER, across 3 seeds: (a) success rate over training; (b) sampling duration and wall-clock time to threshold.

6.2.5 Sphere

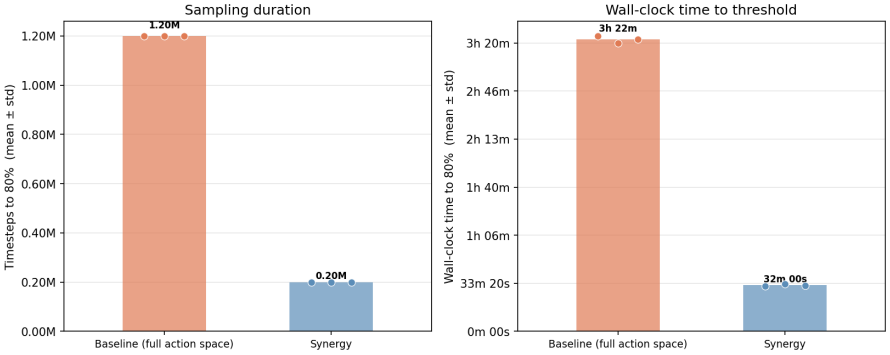
The sphere task yields the most striking result of the entire evaluation. The synergy-constrained agent learns extremely rapidly, reaching near-perfect performance well before 0.50×10^6 timesteps, while the full-action-space baseline follows a much slower trajectory. Both agents eventually reach the absolute 80% success threshold, so we use that level for the comparison. The learning curves are shown in Figure 6.6.

The synergy-constrained agent reaches the 80% threshold in only 0.20×10^6 timesteps (32 m 00 s), compared to 1.20×10^6 timesteps (3 h 22 m) for the baseline — a reduction of approximately 83% in sampling duration and 84% in wall-clock time, corresponding to a roughly six-fold acceleration. We attribute this to the high symmetry of the spherical geometry: because a sphere presents an identical contact surface in every orientation, the reorientation task is intrinsically simpler than for objects with edges or flat faces. The synergy basis provides an immediately

effective exploration prior, and the reduced dimensionality of the action space allows the policy to exploit this prior with very few samples. The sphere task therefore represents an upper bound on the benefit that the synergy parameterisation can confer within this benchmark.



(a) Success rate over training.



(b) Sampling duration and wall-clock time to threshold.

Figure 6.6 Sphere case. Comparison of baseline (full action space) and synergy-reduced SAC+HER, across 3 seeds: (a) success rate over training; (b) sampling duration and wall-clock time to threshold.

Chapter 7

Conclusion and future work

7.1 Conclusion

This thesis investigated whether motor synergies extracted from a single source manipulation task can accelerate reinforcement learning on a set of novel target tasks involving objects with substantially different geometries. We proposed the *Motor Synergy Generalization Framework*, a three-stage pipeline in which a full-DoF SAC+HER agent is first trained on a standard cube reorientation task (Stage A), spatial synergies are then extracted offline via Principal Component Analysis from the joint-action trajectories of the converged policy (Stage B), and the resulting synergy basis is finally frozen and reused as a fixed linear decoder for training new agents on five target tasks (Stage C). The synergy basis is never re-fitted on the target tasks, which turns the pipeline from a dimensionality-reduction technique into a genuine generalisation mechanism: the same low-dimensional motor vocabulary is reused to accelerate the learning of policies for entirely new objects.

The experimental evaluation on the Shadow Dexterous Hand in MuJoCo compared a synergy-constrained agent operating in a 5-dimensional latent space against a full-action-space baseline operating in the native 20-dimensional joint space, across five object geometries: a small cube, a large cube, an egg, a cylinder, and a sphere. The central finding is that constraining the action space to the synergy subspace extracted from a single source task consistently reduces the sampling duration and wall-clock time required to reach a given level of performance across all target objects, regardless of their geometry. The synergy-constrained agent reached comparable or higher final success rates with substantially fewer environment timesteps and less wall-clock training time on every task, with speedups ranging from $\sim 2\times$ on the cubes to $\sim 6\times$ on the sphere, demonstrating that the synergy-constrained approach is significantly more efficient than the full-action-space baseline. This consistent advantage across objects that were never seen during synergy extraction demonstrates that the extracted synergy basis acts as a geometry-agnostic motor prior: a shared coordination structure that transfers across shapes and provides an effective starting point for exploration on any new manipulation task on the same hand.

7.2 Future work

The results of this thesis open several directions for future investigation.

Evaluation without tactile sensing. All experiments in this thesis used the touch-sensor variant of each environment, augmenting the observation with 92 continuous tactile readings distributed across the hand surface. Due to time constraints, the sensor-free variant was not evaluated. Comparing the two settings would clarify the role of tactile feedback in the synergy-constrained regime: it is possible that the reduced motor resolution imposed by the low-dimensional action space makes the agent more reliant on contact information than the full-action-space baseline, or conversely that the synergy prior is robust enough to compensate for the absence of tactile feedback.

Alternative synergy extraction methods. This thesis used PCA as the synergy extraction algorithm, motivated by its closed-form solution, its natural alignment with the postural synergy framework, and the difficulties encountered with NMF and ICA in preliminary experiments. A systematic comparison of PCA with Non-Negative Matrix Factorisation and Independent Component Analysis, under a controlled experimental protocol, would clarify whether the choice of factorisation method has a significant impact on transfer performance, and whether the superiority of PCA observed in our setting generalises to other tasks and platforms.

Adaptive synergy parameterisation. The present framework fixes both the synergy matrix W and the number of components K before Stage C begins. Both could be made adaptive. On one hand, allowing a slow fine-tuning of W during Stage C would preserve the generalisation advantage of the frozen initialisation while recovering expressiveness for the specific target geometry. On the other hand, an adaptive or task-dependent choice of K, possibly informed by the geometry of the target object, could further improve the trade-off between dimensionality reduction and motor expressiveness.

Larger and more diverse object sets. Extending the benchmark to a larger and more diverse set, including irregular and articulated objects, would provide stronger evidence for the geometry-agnosticism hypothesis suggested by the egg and sphere results.

Multi-source synergy extraction. Combining trajectories from multiple source tasks before applying PCA, or learning a shared synergy basis from a play-phase policy that encounters a variety of objects, could produce a richer prior that improves transfer across an even wider range of geometries, along the lines of the SAR approach of Berg et al. [2].

Nonlinear latent action spaces. Replacing the linear PCA decoder with a learned nonlinear mapping could capture the intrinsic low-dimensional manifold of

dexterous manipulation more accurately, potentially improving both final performance and transfer across tasks.

Sim-to-real transfer. Validating the framework on a physical Shadow Hand is the most important next step towards practical applicability. The synergy basis is a compact representation that could be combined with domain randomisation or system identification to bridge the reality gap.

Bibliography

[1] Y. Huang et al. Human-like dexterous manipulation for anthropomorphic five-fingered hands: A review. *Biomimetric Intelligence and Robotics*, 2025.

[2] C. Berg, V. Caggiano, and V. Kumar. SAR: Generalization of physiological agility and dexterity via synergistic action representation. *Autonomous Robots*, 48(8):28, 2024. doi: 10.1007/s10514-024-10182-4.

[3] A. Fukunishi, K. Kutsuzawa, D. Owaki, and M. Hayashibe. Synergy quality assessment of muscle modules for determining learning performance using a realistic musculoskeletal model. *Frontiers in Computational Neuroscience*, 18:1355855, 2024. doi: 10.3389/fncom.2024.1355855.

[4] Farama Foundation. Gymnasium-robotics, 2023. Accessed 2025.

[5] K. Kutsuzawa and M. Hayashibe. Motor synergy generalization framework for new targets in multi-planar and multi-directional reaching task. *Royal Society Open Science*, 9(5):211721, 2022.

[6] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22, 2021.

[7] M. Al Borno, J. L. Hicks, and S. L. Delp. The effects of motor modularity on performance, learning and generalizability in upper-extremity reaching: a computational analysis. *Journal of the Royal Society Interface*, 17(167):20200011, 2020.

[8] M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.

[9] J. Chai and M. Hayashibe. Motor synergy development in high-performing deep reinforcement learning algorithms. *IEEE Robotics and Automation Letters*, 2020.

[10] Z. Deng and J. W. Zhang. Learning synergies based in-hand manipulation with reward shaping. *CAAI Transactions on Intelligence Technology*, 5(3):141–149, 2020.

[11] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[12] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1582–1591, 2018.

[13] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.

[14] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018. doi: 10.48550/arXiv.1812.05905.

[15] M. Hayashibe and S. Shimoda. Synergetic learning control paradigm for redundant robot to enhance error-energy index. *IEEE Transactions on Cognitive and Developmental Systems*, 10(3):573–584, 2018.

[16] M. Plappert, M. Andrychowicz, A. Ray, B. McGrew, B. Baker, G. Powell, J. Schneider, J. Tobin, M. Chociej, P. Welinder, V. Kumar, and W. Zaremba. Multi-goal reinforcement learning: Challenging robotics environments and request for research. *arXiv preprint arXiv:1802.09464*, 2018.

[17] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT Press, Cambridge, MA, USA, 2nd edition, 2018.

[18] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[19] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *CoRR*, abs/1509.02971, 2015.

[20] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

[21] T. Schaul, D. Horgan, K. Gregor, and D. Silver. Universal value function approximators. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 1312–1320, 2015.

[22] M. G. Catalano, G. Grioli, E. Farnioli, A. Serio, C. Piazza, and A. Bicchi. Adaptive synergies for the design and control of the pisa/iit softhand. *The International Journal of Robotics Research*, 33(5):768–782, 2014.

[23] E. Bizzi and V. C. K. Cheung. The neural origin of muscle synergies. *Frontiers in Computational Neuroscience*, 7:51, 2013.

[24] E. Rückert and A. d'Avella. Learned parametrized dynamic movement primitives with shared synergies for controlling robotic and musculoskeletal systems. *Frontiers in Computational Neuroscience*, 7:138, 2013.

[25] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.

[26] A. Bicchi, M. Gabiccini, and M. Santello. Modelling natural and artificial hands with synergies. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 366(1581):3153–3161, 2011.

[27] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[28] M. T. Ciocarlie and P. K. Allen. Hand posture subspaces for dexterous robotic grasping. *The International Journal of Robotics Research*, 28(7):851–867, 2009.

[29] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, planning and control*. Springer, London, UK, 2009.

[30] M. C. Tresch, V. C. K. Cheung, and A. d'Avella. Matrix factorization algorithms for the identification of muscle synergies: Evaluation on simulated and experimental data sets. *Journal of Neurophysiology*, 95(4):2199–2212, 2006.

[31] A. d'Avella and E. Bizzi. Shared and specific muscle synergies in natural motor behaviors. *Proceedings of the National Academy of Sciences of the United States of America*, 102(8):3076–3081, 2005.

[32] A. d'Avella, P. Saltiel, and E. Bizzi. Combinations of muscle synergies in the construction of a natural motor behavior. *Nature Neuroscience*, 6(3):300–308, 2003.

[33] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning*, pages 278–287, 1999.

[34] M. Santello, M. Flanders, and J. F. Soechting. Postural hand synergies for tool use. *Journal of Neuroscience*, 18(23):10105–10115, 1998.

[35] N. A. Bernstein. *The Co-ordination and Regulation of Movements*. Pergamon Press, Oxford, UK, 1967.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Prof. Alessandro De Luca, for giving me the extraordinary opportunity to carry out part of this research at Tohoku University in Japan, and for his continuous feedback and support throughout the development of this work.

I am deeply grateful to my co-supervisor, Prof. Mitsuhiro Hayashibe, for welcoming me into his laboratory and for his dedicated guidance and invaluable support during the four months I spent in Japan. Working in his group was an experience that shaped me both as a person and as a professional, and one I will carry with me throughout my career.

I would also like to thank my labmates at Tohoku University and at Sapienza University of Rome for their support, the stimulating discussions, and the friendly environment they created. Their presence made this journey significantly more enjoyable.

Finally, I am deeply grateful to my family and friends for their constant encouragement and patience during these years of study. This work would not have been possible without their support.